



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра игровой индустрии

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

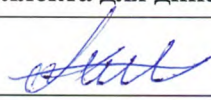
И.О. заведующего
кафедрой _____ П.В. Беляев

« 03 » 06 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по направлению подготовки бакалавров 09.03.04 Программная инженерия

На тему: Компьютерное приложение — игра в жанре «Tower Defence» с применением
искусственного интеллекта для динамической корректировки сложности

Обучающийся


подпись

Ахмин Максим Сергеевич
Фамилия, имя, отчество

шифр 21И1732
группа ИКБО-03-21

Руководитель работы


подпись

к.т.н., доцент,
доцент

Алпатов А.Н.

Консультант


подпись

к.э.н., доцент

Жемерикин О.И.

Москва 2025 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий

Кафедра игровой индустрии

СОГЛАСОВАНО

И.О. заведующего
кафедрой

подпись

Беляев Павел Вячеславович

«10» марта 2025 г.

УТВЕРЖДАЮ

Директор
института

подпись

Зуев Андрей Сергеевич

«10» марта 2025 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра

Обучающийся

Ахмин Максим Сергеевич

Фамилия Имя Отчество

Шифр

21И1732

Направление
подготовки

09.03.04

индекс направления

Программная инженерия

наименование направления

Группа

ИКБО-03-21

1. Тема выпускной квалификационной работы: Компьютерное приложение – игра в жанре «Tower Defence» с применением искусственного интеллекта для динамической корректировки сложности

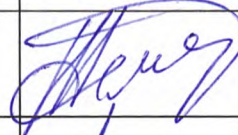
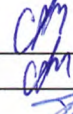
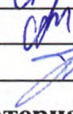
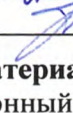
2. Цель и задачи выпускной квалификационной работы

Цель работы: спроектировать и разработать игру в жанре «Tower Defence» с применением искусственного интеллекта для динамической корректировки сложности

Задачи работы: провести анализ предметной области, в том числе конкурентных продуктов; формализовать задачи на проектирование и разработку игры; спроектировать игру (функциональную схему, концепт-документ, гейм-дизайн документ, архитектуру); определить и обосновать технические и программные средства для разработки игры; разработать компьютерную игру в жанре «Tower Defence» с применением искусственного

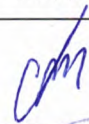
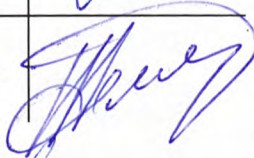
интеллекта для динамической корректировки сложности; произвести тестирование видеоигрового продукта; рассчитать экономическую эффективность и стоимость проведения работ; оформить пояснительную записку согласно ГОСТ 7.32-2017

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Исследовательский раздел		22.03.2025
1.1	Анализ существующих игр в жанре «Tower Defence» на текущем рынке		
1.2	Концепт идеи игрового продукта		
1.3	Постановка цели и задач на разработку игрового продукта		
2	Проектный раздел		22.03.2025
2.1	Разработка гейм-дизайн документа		
2.2	Выбор и обоснование стека технологий		
2.3	Проектирование архитектуры игрового продукта		
2.4	Проектирование модуля динамической корректировки сложности		
2.5	Системные требования игрового продукта		
3	Технологический раздел		19.04.2025
3.1	Разработка игрового продукта		
3.2	Разработка модуля динамической корректировки сложности		
3.3	Интеграция модуля в игровой процесс		
3.4	Тестирование игрового продукта		
4	Экономический раздел		16.05.2025
4.1	Организация и планирование работ по теме		
4.2	Расчет стоимости проведения работ по теме		
5	Введение, заключение, список источников, приложения		17.05.2025
6	Презентационный материал, аннотация		26.05.2025
7	Нормоконтроль		27.05.2025

4. Перечень разрабатываемых документов и графических материалов: электронная версия выпускной квалификационной работы бакалавра, презентационный материал с основными результатами выпускной квалификационной работы бакалавра

5. Руководитель и консультант выпускной квалификационной работы

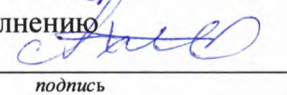
Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	доцент	Алпатов Алексей Николаевич	
Консультант по экономическому разделу	доцент	Жемерикин Олег Игоревич	

Задание выдал
Руководитель ВКР:


подпись

«10» марта 2025 г.

Задание принял к исполнению
Обучающийся:


подпись

«10» марта 2025 г.

УДК 004.4

Руководитель ВКР: к.т.н., доцент А.Н. Алпатов

Консультант по экономическому разделу: к.э.н., доцент О.И. Жемерикин

Ахмин М.С., Выпускная квалификационная работа по образовательной программе «Разработка и дизайн компьютерных игр и мультимедийных приложений» направления подготовки «Программная инженерия» на тему «Компьютерное приложение – игра в жанре «Tower Defence» с применением искусственного интеллекта для динамической корректировки сложности»: М. 2025 г., МИРЭА – Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра игровой индустрии (ИИ) – 56 стр., 19 рис., 8 табл., 7 лист., 31 источн. (в т.ч. 10 на английском яз.), 4 прил.

Ключевые слова: игровая индустрия, игровой дизайн, tower defence, сложность игры, искусственный интеллект, динамическая корректировка сложности, обучение с подкреплением.

Целью работы является проектирование и разработка игры в жанре «Tower Defence» с применением искусственного интеллекта для динамической корректировки сложности.

Akhmin M.S., Final qualification work for the educational program “Development and design of computer games and multimedia applications” of the training program “Software engineering on the topic “A computer game in the genre of “Tower Defense” using artificial intelligence to dynamically difficulty adjustment”: M. 2025, MIREA - Russian Technological University (RTU MIREA), Institute of Information Technologies (IIT), Department of gaming industry (Department of GI) - 56 p., 19 ill., 8 tabl., 7 list., 31 ref. (inc. 10 in English), 4 apps.

Keywords: gaming industry, game design, tower defence, game complexity, artificial intelligence, dynamic difficulty adjustment, reinforcement learning.

The goal of the work is to design and develop a game in the “Tower Defence” genre using artificial intelligence to dynamically difficulty adjustment.

РТУ МИРЭА: 119454, Москва, пр-т Вернадского, д. 78
кафедра игровой индустрии (ИИ)

Тираж: 1 экз. (на правах рукописи)

Файл: «090304_21И1732_Ахмин МС.docx», исполнитель Ахмин М.С.

© М.С. Ахмин

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ.....	6
ВВЕДЕНИЕ	7
1 Исследовательский раздел.....	10
1.1 Анализ существующих игр в жанре «Tower Defence» на текущем рынке.....	10
1.1.1 История и эволюция жанра TD	10
1.1.2 Типы TD-игр	10
1.1.3 Анализ игровых механик и особенностей жанра	11
1.1.4 Вывод.....	12
1.2 Концепт идеи игрового продукта.....	12
1.3 Постановка цели и задач на разработку игрового продукта	12
1.4 Вывод к разделу 1	13
2 Проектный раздел	14
2.1 Разработка гейм-дизайн документа	14
2.2 Выбор и обоснование стека технологий.....	14
2.2.1 Игровой движок	14
2.2.2 Язык программирования модуля динамической корректировки сложности	15
2.2.3 Графика и хранение данных.....	15
2.3 Проектирование архитектуры игрового продукта	15
2.3.1 Выбор используемых технологий моделирования	15
2.3.2 Моделирование структурной схемы.....	16
2.3.3 Моделирование функциональной схемы.....	17
2.4 Проектирование модуля динамической корректировки сложности ...	19
2.4.1 Общее описание модуля.....	19
2.4.2 Моделирование функциональной схемы.....	19
2.5 Системные требования игрового продукта	20
2.6 Вывод к разделу 2	20
3 Технологический раздел	21
3.1 Разработка игрового продукта	21
3.1.1 Разработка графического и звукового оформления игры.....	21
3.1.2 Разработка игрового уровня	23

3.1.3	Разработка системы сохранений	27
3.1.4	Разработка интерфейсных сцен	27
3.2	Разработка модуля динамической корректировки сложности	29
3.3	Интеграция модуля в игровой процесс	32
3.3.1	Создание и обучение модели симуляции игрока	32
3.3.2	Обучение модели модуля динамической корректировки сложности	33
3.4	Тестирование игрового продукта	34
3.5	Вывод к разделу 3	35
4	Экономический раздел	36
4.1	Организация и планирование работ по теме	36
4.1.1	Организация работ	36
4.1.2	График проведения работ	40
4.2	Расчет стоимости проведения работ	40
4.2.1	Статья «Сырье и материалы»	41
4.2.2	Статья «Основная заработная плата»	42
4.2.3	Статья «Дополнительная заработная плата»	43
4.2.4	Статья «Страховые взносы»	45
4.2.5	Статья «Амортизация»	45
4.2.6	Статья «Прочие расходы»	47
4.2.7	Калькуляция затрат по проекту	48
4.3	Вывод к разделу 4	49
	ЗАКЛЮЧЕНИЕ	50
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	52
	ПРИЛОЖЕНИЕ А	57
	ПРИЛОЖЕНИЕ Б	61
	ПРИЛОЖЕНИЕ В	69
	ПРИЛОЖЕНИЕ Г	74

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

В настоящем отчёте применяют следующие термины с соответствующими определениями.

ИИ — Искусственный Интеллект

ПК — Персональный Компьютер

DDA — Dynamic Difficulty Adjustment, динамическая корректировка сложности

RAM — Random Access Memory, оперативная память

TD — Tower Defence, поджанр стратегических игр в реальном времени

UI — User Interface, пользовательский интерфейс

ВВЕДЕНИЕ

Сегодня компьютерные игры — популярный способ развлечения. Многие играют, чтобы отвлечься, расслабиться, отдохнуть и весело провести время. Объём игрового рынка за 2024 год составил 184,3 миллиарда долларов, что говорит о большой популярности этой индустрии [1]. А развитие технологий искусственного интеллекта открыло новые возможности для создания динамически изменяемых игровых механик.

Актуальность выбранной темы обоснована стремительным развитием искусственного интеллекта за последнее время. Использование данных технологий в игровой индустрии может привести к разработке успешного продукта.

Новизна работы заключается в использовании искусственного интеллекта для динамической корректировки сложности игровых параметров в зависимости от мастерства игрока. Данный подход должен обеспечить максимальную вовлеченность игрока, так как на нее в основном влияют именно сложность или легкость прохождения игры [2, 3].

Объектом исследования являются игры в жанре «Tower Defence». Предметом исследования — использование ИИ для динамической корректировки сложности игрового процесса.

Гипотеза работы — использование ИИ для динамической корректировки сложности обеспечивает большую вовлеченность игрока.

Целью работы является проектирование и разработка игры в жанре «Tower Defence» с применением искусственного интеллекта для динамической корректировки сложности. Для достижения поставленной цели необходимо решить следующие задачи:

- Провести анализ жанра «Tower Defence».
- Разработать гейм-дизайн документ.
- Произвести выбор и обоснование стека технологий.

- Спроектировать игровой продукт и модуль динамической корректировки сложности.
- Разработать игровой продукт.
- Разработать и интегрировать в игровой продукт модуль динамической корректировки сложности.
- Выполнить тестирование разработанного продукта.
- Осуществить расчет экономической эффективности и стоимость проведения работ.

Выпускная квалификационная работа состоит из четырех основных разделов. В исследовательском разделе производится анализ существующих игр жанра TD, изучаются основные игровые механики. В проектном разделе разрабатывается гейм-дизайн документ, а также архитектура игрового продукта и модуля динамической корректировки сложности. В технологическом разделе описывается разработка игрового продукта и модуля динамической корректировки сложности и тестирование разработанного функционала. В экономическом разделе описывается организация и расчет стоимости проведения работ.

В процессе написания выпускной квалификационной работы автор руководствовался локальными и глобальными актами, связанными со спецификацией образовательной программы [4-8], а также следующими нормативными актами:

1. «О защите населения и территории от чрезвычайных ситуаций природного и техногенного характера» от 21.12.1994 № 68-ФЗ.
2. «Об основах охраны здоровья граждан в Российской Федерации» от 21.11.2011 № 323-ФЗ.
3. «О гражданской обороне» от 12.02.1998 № 28-ФЗ.
4. Приказ Минздравсоцразвития РФ от 04.05.2012 № 477н «Об утверждении перечня состояний, при которых оказывается первая помощь, и перечня мероприятий по оказанию первой помощи».

5. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ.
6. СанПиН 2.2.2/2.4.1340-03 «Гигиенические требования к персональным электронно-вычислительным машинам и организации работы».

1 Исследовательский раздел

1.1 Анализ существующих игр в жанре «Tower Defence» на текущем рынке

1.1.1 История и эволюция жанра TD

Tower Defense (с англ. – «защита башен») [9] – поджанр стратегических видеоигр в реальном времени, в котором от игрока требуется защищать какой-либо объект от наступающих врагов с помощью строительства оборонительных укреплений или башен.

Прототипом жанра считается игра Rampart от Atari Games, вышедшая в 1990 году. В ней впервые был реализован один из основных элементов игрового процесса типичной игры Tower Defence – игроку предлагалось строить защитные сооружения для отражения волн наступающих противников.

В окончательном виде поджанр сложился в середине 2000-х годов, когда соответствующие механики были популяризированы модами и дополнительными картами к таким играм, как Starcraft и Warcraft III: Reign of Chaos от студии Blizzard Entertainment.

С середины 2000-х годов жанр быстро развивался во многом благодаря распространению мобильных устройств и портативных игровых систем.

1.1.2 Типы TD-игр

Существует множество типов игры в жанре TD. Ниже перечислены некоторые из них:

- Классический TD – игрок самостоятельно строит башни, а враги следуют по фиксированному пути.
- Maze TD – игрок самостоятельно строит лабиринт из башен, усложняя путь врагам.
- Tower Offence – вместо обороны цель игрока самому пробиться через некоторую оборону.

– Гибриды – смесь TD с другими жанрами (например, игра Mindustry, которая является гибридом жанров «Tower Defence» и «Строительство базы»).

1.1.3 Анализ игровых механик и особенностей жанра

Для выполнения анализа жанра, приведем наиболее популярные игры данного направления. В таблице 1.1 представлена сводная информация по некоторым популярным играм в жанре TD.

Таблица 1.1 – Сравнительная таблица игр жанра «Tower Defence» [разработано автором]

Игра, год выхода	Рейтинг	Механики	Адаптация сложности	Минусы
Kingdom Rush, 2014	97%	Фиксированные дорожки, геройские способности	Нет динамической сложности	Линейный баланс уровней
Bloons TD 6, 2018	97%	Разнообразные башни, кооп-режим, случайные модификации	Наличие режимов сложности	Иногда резкий баланс сложности
Mindustry, 2019	95%	TD + Строительство базы, автоматизация	Враги подстраиваются под слабые места	Большой упор под строительство базы и автоматизацию, нежели TD
Infinitode 2, 2021	91%	Свободное размещение башен, сложные механики	Враги получают случайные улучшения	Высокий порог входа

После обзора популярных игр в жанре «Tower Defence» можно сформировать список механик и особенностей, присущие большинству игр жанра, который представлен ниже.

– Структура карты: враги двигаются по какому-то лабиринту игрока или фиксированному маршруту; точки появления врагов также могут быть либо фиксированными, либо случайными.

– Башни: игрок ставит башни для уничтожения врагов; различные башни выполняют различные функции и добавляют разнообразия к геймплею.

– Прогрессия: за прохождение волн игрок получает ресурсы, необходимые для постройки новых и улучшения существующих башен. В

некоторых играх система улучшения башен выглядит линейно, в некоторых она представляет собой ветвистую систему.

- Типы врагов: в игре могут присутствовать различные типы врагов, которые имеют различные характеристики.
- Сложность: в большинстве игр враги имеют кривую сложности, зависимость от времени или от номера волны.

1.1.4 Вывод

Популярные игры жанра TD имеют довольно различные механики и различные способы адаптации сложности. Однако все игры ставят перед игроком похожие цели, каждая из игр обладает некоторым разнообразием башен и врагов.

Таким образом, проанализированные игры показывают, что динамическая корректировка сложности, тем более на основе ИИ, используется редко или недостаточно гибко. В связи с этим данная работа направлена на разработку TD-игры с более проработанной адаптацией сложности, что выделит ее на фоне существующих проектов [10].

1.2 Концепт идеи игрового продукта

Dynamic Defence — 2D игра для ПК на Windows в жанре «Tower Defence» в минималистической графике. Игра представляет собой набор уровней с различными расстановками, для прохождения которых необходимо продержаться некоторое количество волн. В игре представлено несколько типов башен и врагов. Ключевой особенностью игры является динамическая корректировка сложности уровня в процессе прохождения, что в теории должно обеспечить наиболее оптимальный игровой опыт игроку [11].

1.3 Постановка цели и задач на разработку игрового продукта

Целью разработки проекта является создание игры в жанре «Tower Defence» с применением искусственного интеллекта для динамической корректировки сложности.

Ниже перечислен список основных задач, которые должны быть выполнены для достижения поставленной цели.

1. Разработка гейм-дизайн документа: определение ключевых механик игры (типы врагов, башен, улучшения), определение визуального стиля и атмосферы игры.
2. Выбор и обоснование стека технологий: определение игрового движка, языков программирования и библиотек, наиболее подходящие для реализации задуманного продукта.
3. Проектирование архитектуры проекта: проектирование архитектуры игрового продукта и модуля DDA.
4. Разработка игрового продукта: разработка игровых механик и геймплея; создание или поиск в открытых источниках спрайтов и звукового сопровождения.
5. Разработка и интеграция модуля динамической корректировки сложности: разработка алгоритмов анализа поведения игрока; реализация механизмов изменения сложности.
6. Тестирование и оптимизация: проведение тестов на стабильность работы игры, отладка ошибок и исправление багов, оптимизация производительности.

В результате работы должен быть создана полноценная версия игры, в которую интегрирован модель динамической корректировки сложности в зависимости от мастерства игрока.

1.4 Вывод к разделу 1

В результате исследовательского этапа была проведена характеристика жанра Tower Defence и выполнен его анализ. Рассмотрены наиболее популярные игры данного направления, на основе которых сформированы обобщённые сведения о ключевых аспектах жанра. Сформулирована концепция разрабатываемого продукта, а также определены цель и задачи проекта.

2 Проектный раздел

2.1 Разработка гейм-дизайн документа

Гейм-дизайн документ является основным проектным документом, в котором фиксируются все ключевые аспекты разработки игры. Он включает в себя описание игровых механик, геймплея и геймплейного цикла, графического и звукового оформления, а также наполнение пользовательского интерфейса.

Сам гейм-дизайн документ, разработанный в рамках данного проекта, представлен в Приложении А.

2.2 Выбор и обоснование стека технологий

2.2.1 Игровой движок

Проведем сравнительный анализ наиболее популярных игровых движков, а именно Unity [12], Unreal Engine 5 и Godot. Результат представлен в таблице 2.1.

Таблица 2.1 – Сравнительная таблица игровых движков [разработано автором]

Характеристика	Unity	Unreal Engine 5	Godot
Лицензия	Бесплатно до определенного уровня дохода с игры, потом необходимо покупать лицензию	Бесплатно до определенного уровня дохода с игры, потом необходимо платить процент от дохода	Полностью бесплатно
Языки программирования	C#	C++, Blueprints	GDScript, C#, C++
Производительность	Высокая	Высокая, но требовательная к ресурсам	Средняя
Графика	Хорошая	Фотореалистичная	Простая
Возможность 2D разработки	Отличная поддержка 2D	Ограниченная поддержка 2D	Лучшая поддержка 2D
Сообщество и ресурсы	Большое сообщество, много обучающего материала	Очень большое сообщество, но сложное освоение движка	Маленькое сообщество, небольшое количество ресурсов

Проведя сравнительный анализ игровых движков, можно сделать вывод, что для проекта больше подходят Unity и Godot в связи с тем, что отсутствует

необходимость в продвинутой физике и графике, а также в связи с отличной поддержкой 2D у этих движков. Предпочтение было отдано в сторону Unity, так как его сообщество гораздо более обширное. В связи с этим основным языком программирования является C#.

2.2.2 Язык программирования модуля динамической коррективности сложности

Есть несколько способов разработки и интеграции модуля динамической коррективности сложности в игру. Первый – написание ИИ-сервиса на любом подходящем для этого языке программирования, и дальнейшее использование этого сервиса при помощи REST [13] запросов из скриптов C#.

Второй способ – разработка модуля на языке Python, и использование ML-Agents [14] для взаимодействия с Unity. Данный способ позволяет без проблем обучать написанную модель в среде Unity.

Поскольку использование ML-Agents значительно упрощают интеграцию и обучение модели в Unity, был выбран второй способ.

2.2.3 Графика и хранение данных

Основная графика планируется использоваться из открытых источников. При необходимости спрайты и другие элементы интерфейса будут создаваться в Photoshop или аналогичных графических редакторах.

Для хранения конфигураций и прогресса игрока будет использоваться формат JSON [15], так как он удобен для хранения структурированных данных и легко интегрируется с Unity.

2.3 Проектирование архитектуры игрового продукта

2.3.1 Выбор используемых технологий моделирования

Для проектирования архитектуры игрового продукта была использована методология UML [16], а конкретно диаграмма классов для структурной схемы и диаграмма активностей для функциональной схемы. Структурная схема показывает совокупность используемых в игре классов и отношения между ними. Функциональная схема показывает процесс прохождения уровня игроком.

2.3.2 Моделирование структурной схемы

Разработаем диаграмму классов. Для этого опишем основные классы, используемые в игре.

- Enemy – абстрактный класс врага, цель которого дойти до базы игрока. Поскольку в игре несколько типов врагов, они наследуются от класса Enemy и переопределяют его свойства.
- EnemySpawn – класс, отвечающий за вызов волн врагов, представляющие собой экземпляры класса Enemy.
- EnemySpawnPanel – класс, отвечающий за корректное отображение свойств врагов.
- Radius – класс, отвечающий за отображение линии некоторого радиуса вокруг некоторой точки. Используется для визуализации радиуса башен.
- Tower – абстрактный класс башни, реализующая в себе методы поиска врага, поворота башни и атаки. От класса Tower наследуются классы башен, в которых переопределяются свойства и, при необходимости, некоторые методы.
- TowerPanel – класс, отвечающий за корректное отображение свойств башни на экран.
- PlatformPanel – класс, отвечающий за отображение панели покупки башен.
- Platform – класс, отвечающий за работу платформы, а именно корректное отображение панелей (TowerPanel или PlatformPanel).
- Base – класс, хранящий в себе данные о количестве здоровья у базы и количестве монет у игрока.
- CameraMove – класс, отвечающий за передвижение и приближение камеры по уровню.
- DataStruct – класс, описывающий структуру файла сохранения.

– SaveFileHandler – класс, реализующий низкоуровневые методы для прямой загрузки и сохранения данных в файл, используемый в качестве хранилища.

– SaveManager – класс, отвечающий за реализацию методов бизнес-логики, обеспечивающих взаимодействие игры с данными, хранящимися в файле через SaveFileHandler.

Разработанная диаграмма классов представлена на рисунке 2.1.

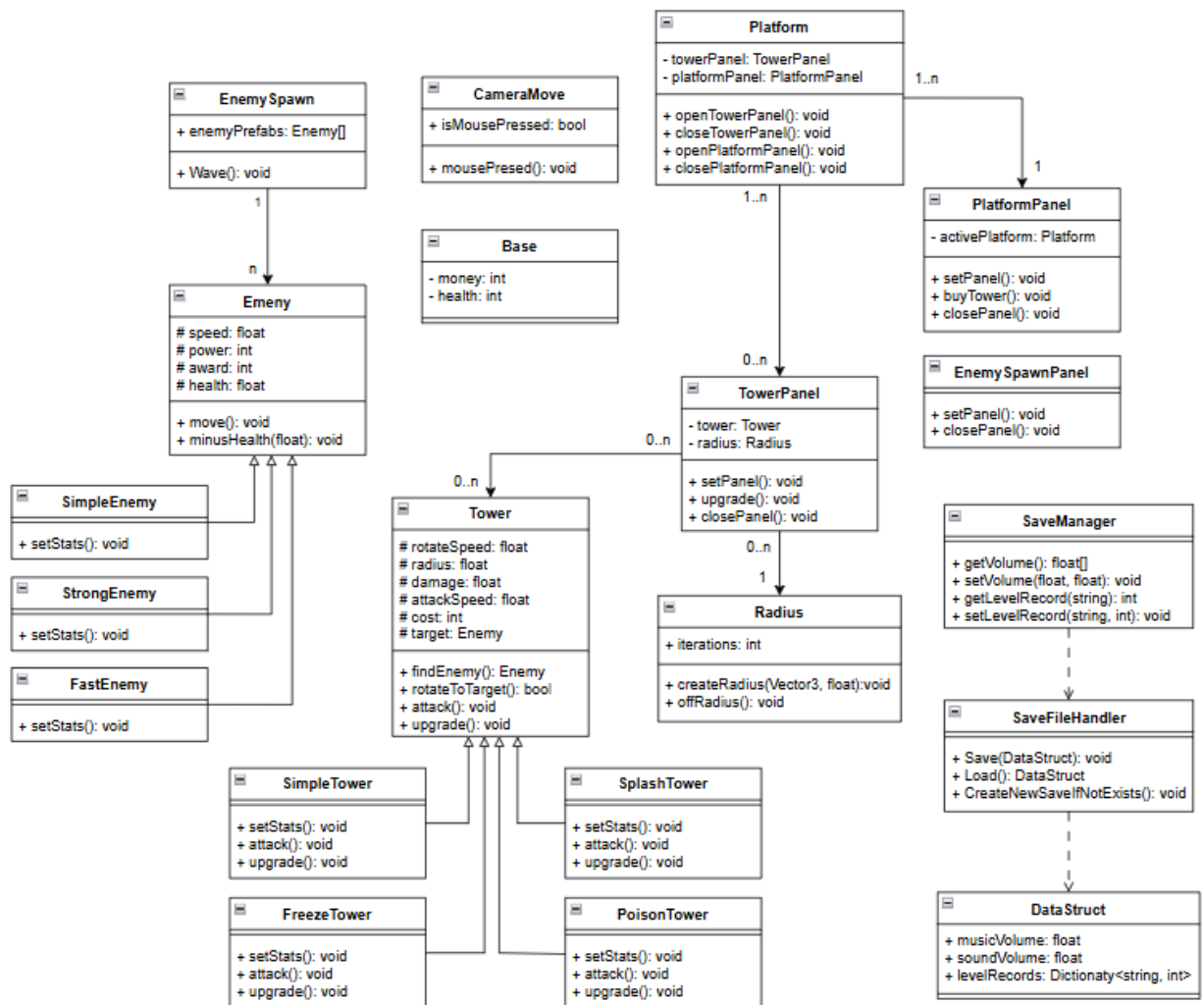


Рисунок 2.1 – Диаграмма классов [разработано автором]

2.3.3 Моделирование функциональной схемы

Разработаем диаграмму активностей, показывающую процесс прохождения уровня.

Процесс начинается с запуска уровня, затем, после начальной расстановки башен игроком, идет запуск волны. Если волна побеждена, игрок

получает монеты, иначе у базы игрока отнимается определенное количество здоровья.

Если у базы достаточное количество здоровья, и, если уровнем предопределены еще волны, цикл начинается заново. Иначе уровень завершается, а результаты сохраняются.

Разработанная диаграмма активностей представлена на рисунке 2.2.

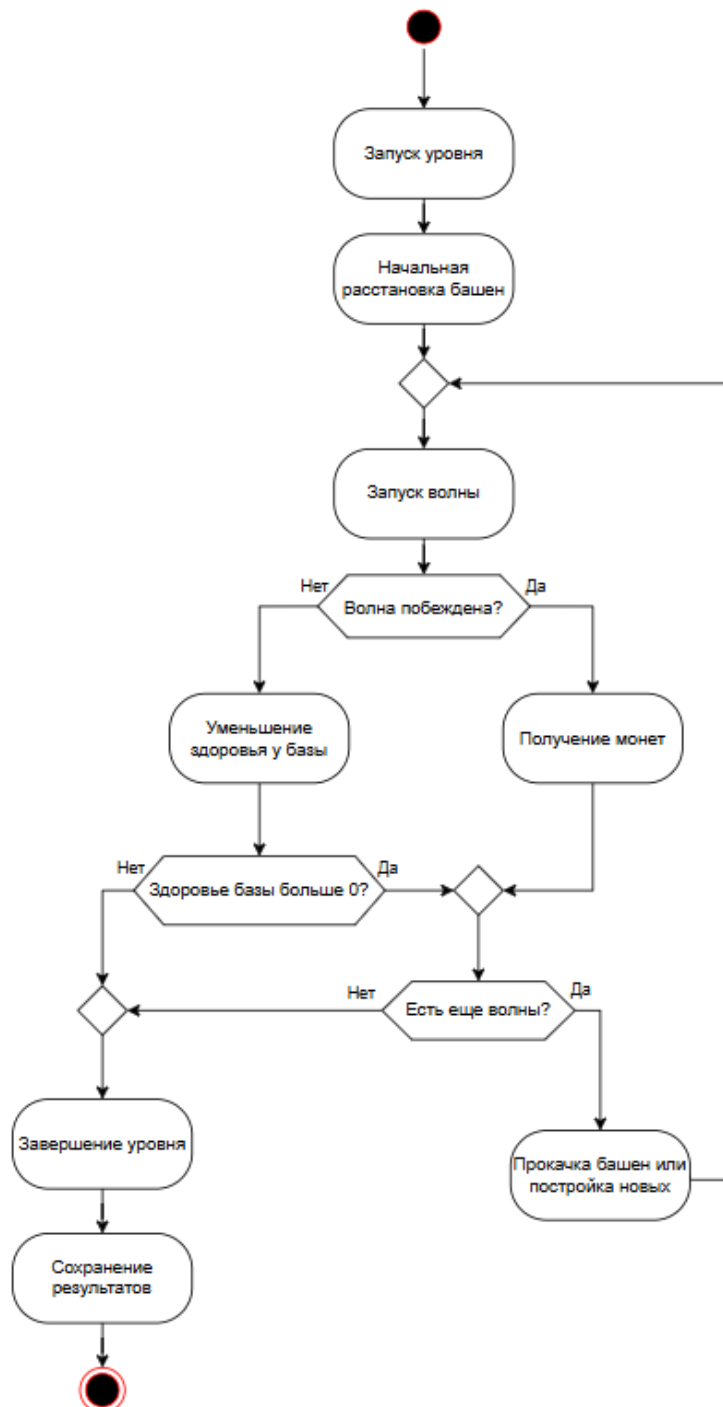


Рисунок 2.2 – Диаграмма активностей [разработано автором]

2.4 Проектирование модуля динамической корректировки сложности

2.4.1 Общее описание модуля

Модуль динамической корректировки сложности – программа, автоматически изменяющая параметры игровой сцены в зависимости от мастерства игрока. Целью модуля является поддержание интереса игрока на протяжении всего игрового процесса. Стоит отметить, что работа модуля должна быть незаметной для игрока, так как в ином случае это чревато непредсказуемостью игрового процесса [17].

Ниже перечислены игровые элементы, которые могут быть изменены с помощью проектируемого модуля:

- Здоровье врагов.
- Количество врагов в волне.
- Награда за успешное завершение волны.

2.4.2 Моделирование функциональной схемы

Разработаем функциональную схему в виде диаграммы активностей нотации UML. Модуль постоянно оценивает уровень мастерства игрока, и в зависимости от него, изменяет параметры игра. Разработанная диаграмма активностей представлена на рисунке 2.3.

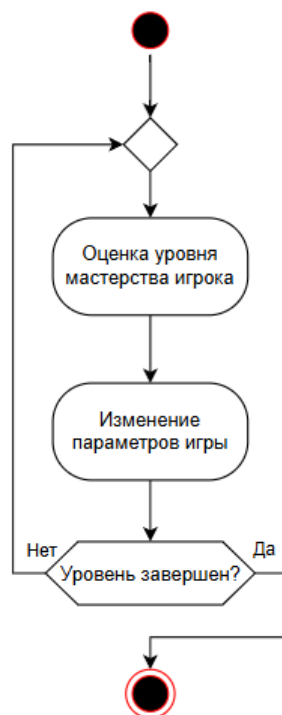


Рисунок 2.3 – Диаграмма активностей модуля динамической корректировки сложности [разработано автором]

2.5 Системные требования игрового продукта

Ниже перечислены минимальные системные требования для ПК.

- Операционная система: Windows 7/8/10/11 64-бит.
- Процессор: Intel Celeron G1840, AMD A6-9500 или лучше (2+ ядер, 2.0 GHz+).
- Оперативная память: 256 МБ ОЗУ.
- Видеокарта: Совместимая с DirectX 10 или OpenGL 3.3, с минимум 256 МБ видеопамяти.
- Место на диске: 150 МБ свободного места.

2.6 Вывод к разделу 2

В рамках проектного этапа был разработан гейм-дизайн документ, определён технологический стек, оптимально соответствующий задачам реализации продукта. Выполнено проектирование игрового приложения и модуля динамической корректировки сложности. Для игрового продукта построены структурная и функциональная схемы, для модуля динамической корректировки сложности – функциональная схема.

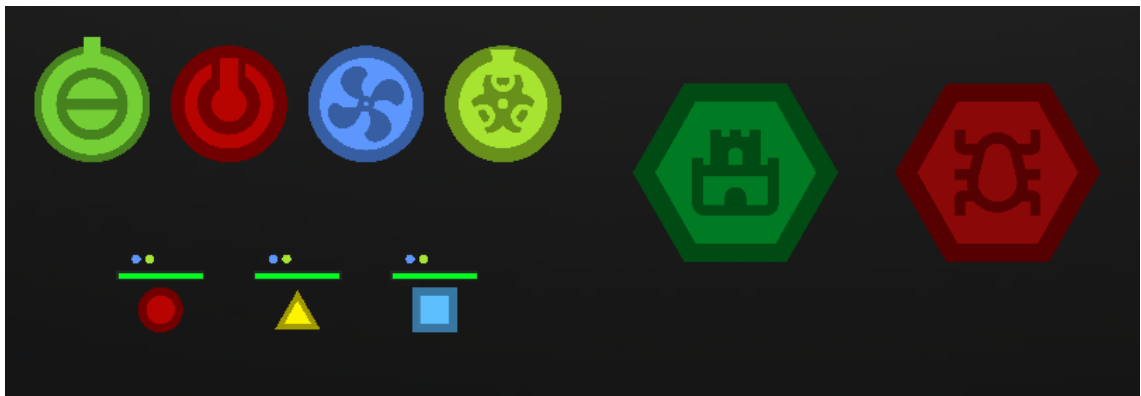


Рисунок 3.2 – Игровые объекты [разработано автором]

Для визуализации выстрелов башен и взрывов в открытых источниках интернета были найдены таблицы спрайтов [18], среди которых были выбраны наиболее подходящие анимации по визуальному стилю. На рисунке 3.3 представлена одна из таблиц спрайтов с выделенными на ней нужными для конкретной анимации спрайтами.

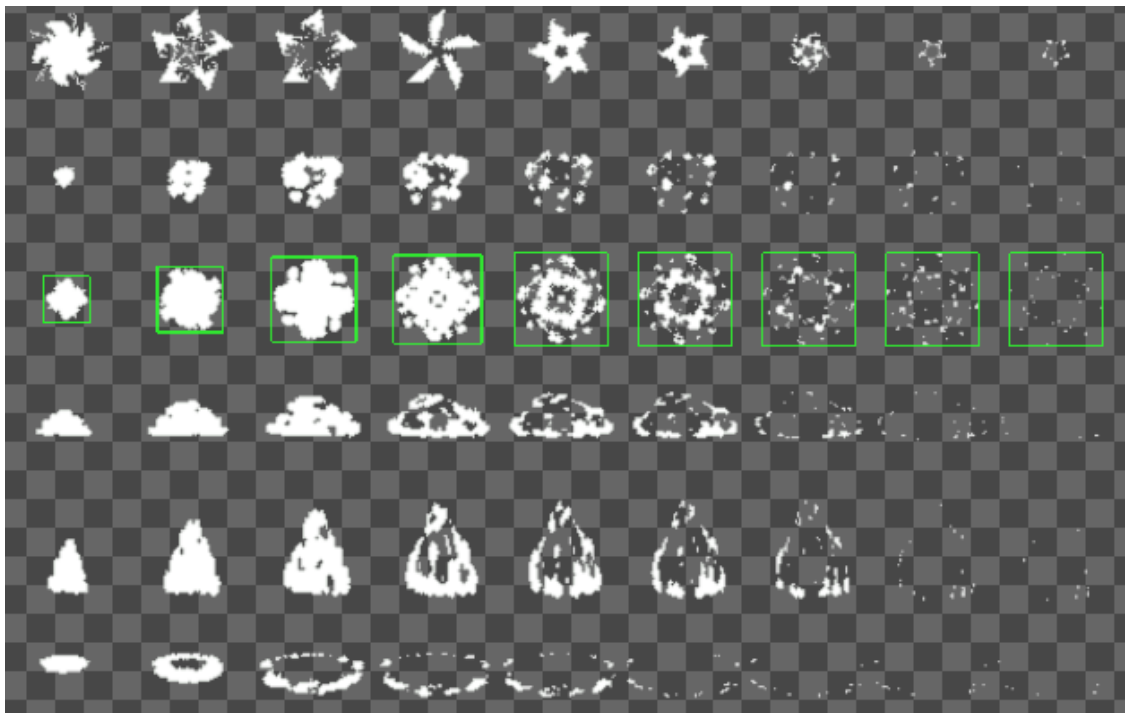


Рисунок 3.3 – Таблица спрайтов [19]

В открытых источниках были найдены необходимые звуки для игры (звук нажатия кнопок, звук улучшения башни и так далее) и набор музыки. Для плавного переключения между музыкальными композициями, был создан класс MusicController. Раз в единицу времени класс плавно изменяет громкость музыки на 0, меняет композицию и возвращает громкость на прежний уровень.

3.1.2 Разработка игрового уровня

Опишем создание основной игровой сцены для игры в Unity. Сам уровень состоит из пути, по которому идут враги и платформ под башни. Путь создается с помощью компонента Line Renderer [20]. На рисунке 3.4 представлен прототип уровня с путем и платформами.

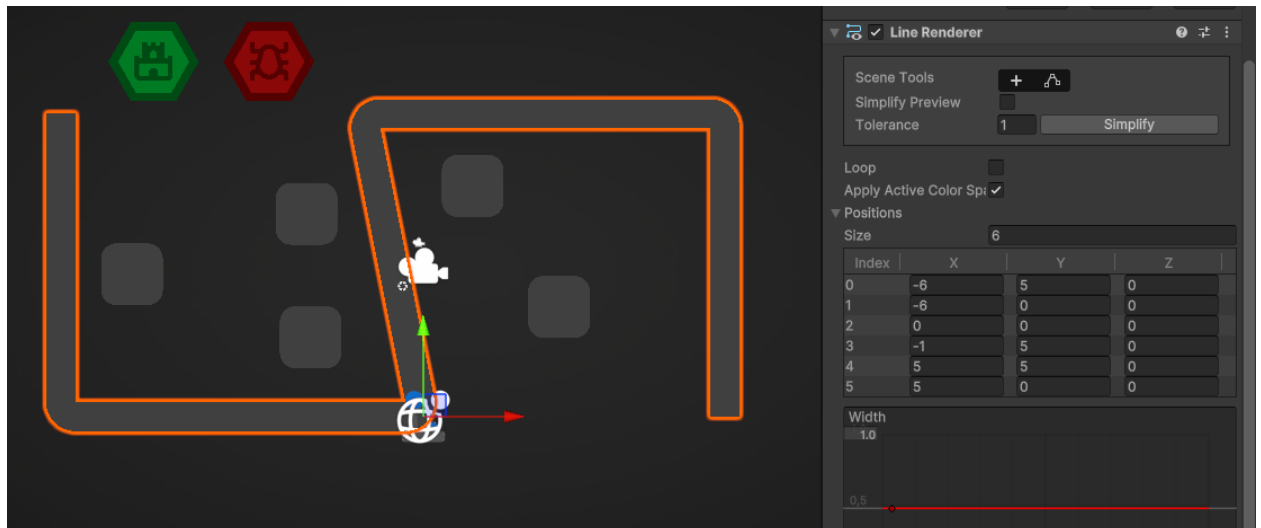


Рисунок 3.4 – Прототип уровня [разработано автором]

Создание нового уровня осуществляется посредством задания нового маршрута, что достигается путем изменения параметров компонента Line Renderer, а также путём перемещения и изменения количества платформ.

Для реализации перемещения пользователя по уровню был создан класс CameraMove. Данный класс перемещает камеру по уровню, если пользователем нажата кнопка мыши, а также приближает и отдаляет камеру в зависимости от прокрутки колесика мыши. Реализация класса представлена в листинге Б.1.

В ходе разработки также был создан класс Base, который хранил в себе значения количества денег игрока и здоровья базы. Класс отвечает за корректное отображение данных величин на экране, а также за окончание уровня.

Был реализован абстрактный класс Enemy, целью которого является перемещение объекта по пути. Поскольку путь представляется набором точек, то класс просто перемещает объект в сторону следующей точки с некоторым

смещением, чтобы враги двигались не ровно друг за другом. Часть класса представлена в листинге 3.1.

Листинг 3.1 – Часть класса Enemy [разработано автором]

```
protected virtual void Update()
{
    Vector3 pos = line.GetPosition(posInd) + new Vector3(epsX,
epsY, 0);
    float distance = Mathf.Pow(
Mathf.Pow(pos.x - transform.position.x, 2) +
Mathf.Pow(pos.y - transform.position.y, 2), 0.5f);
    while (distance < 0.01f * baseSpeed)
    {
        posInd++;
        if (posInd == line.positionCount)
            Destroy(gameObject);
        pos = line.GetPosition(posInd) + new Vector3(epsX,
epsY, 0);
        distance = Mathf.Pow(
Mathf.Pow((pos.x - transform.position.x), 2) +
Mathf.Pow((pos.y - transform.position.y), 2), 0.5f);
    }
    float moveX = (pos.x - transform.position.x) / distance;
    float moveY = (pos.y - transform.position.y) / distance;
    float angle = -Mathf.Asin((pos.x - transform.position.x) /
distance) * 180 / Mathf.PI;
    if (moveY < 0)
    {
        angle = 180 - angle;
    }
    sprite.transform.rotation = Quaternion.Euler(0, 0, angle);

    transform.Translate(new Vector3(moveX, moveY, 0) * speed *
Time.deltaTime);

    liveTime += Time.deltaTime;
}
```

Также был создан класс EnemyHpBar. Его задачей является корректное отображение уровня здоровья врага, а также отображение меток при воздействии морозной или ядовитой башнями.

Для реализации логики башен был создан класс Tower. Данный класс является абстрактным и содержит методы для поиска цели, поворота башни и атаки.

Были созданы классы для управления UI-интерфейсом платформы: PlatformPanel и TowerPanel. Первый из них создается в единичном экземпляре

и отвечает за логику кнопок в панели покупки башни (рисунок 3.5). Второй создается каждый раз при создании новой башни и к ней привязывается, отвечает за отображение и изменение характеристик башен (рисунок 3.6).

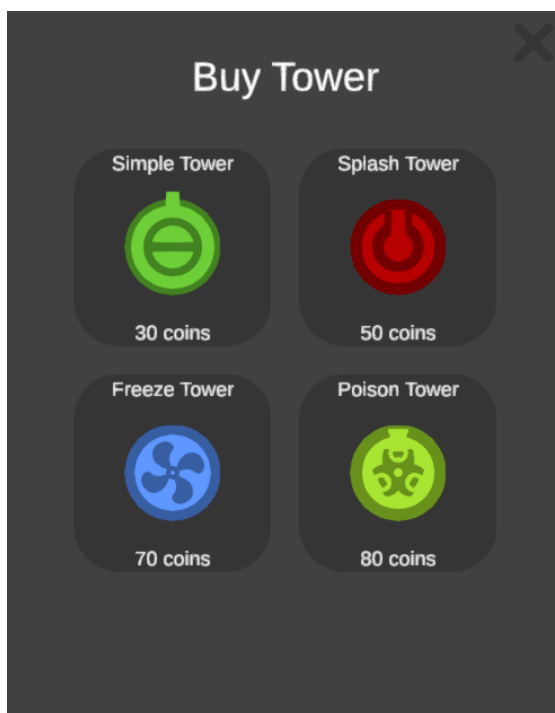


Рисунок 3.5 – Панель покупки башни [разработано автором]

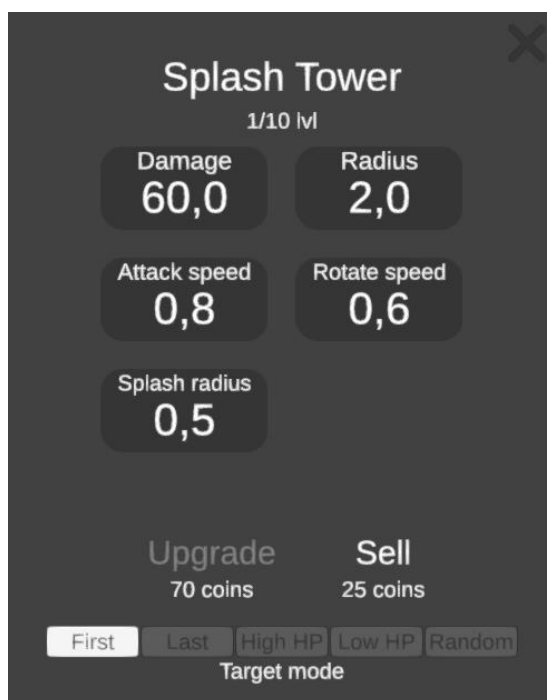


Рисунок 3.6 – Панель башни [разработано автором]

Был реализован класс Platform, задача которого открывать одну из описанных выше панелей исходя из того, стоит ли на платформе башня, или нет.

Затем был создан EnemySpawnPanel. Он отвечает за корректное отображение характеристик врагов на панели вражеской базы (рисунок 3.7).

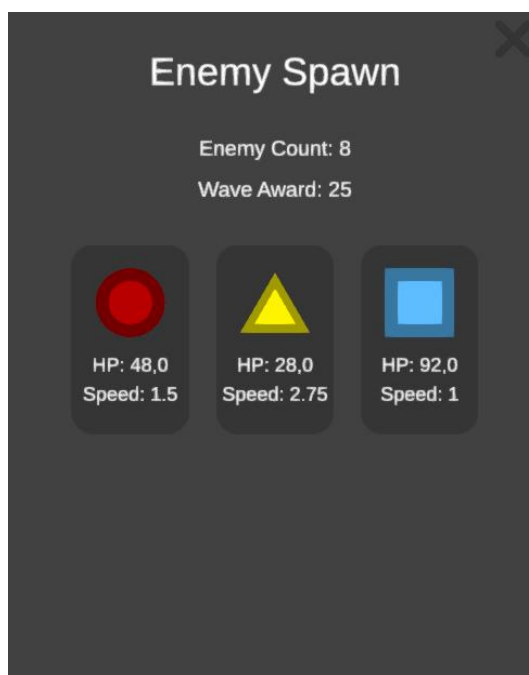


Рисунок 3.7 – Панель вражеской базы [разработано автором]

Также был создан класс PauseController, отвечающий за работу меню паузы и слайдеров изменения громкости (рисунок 3.8).

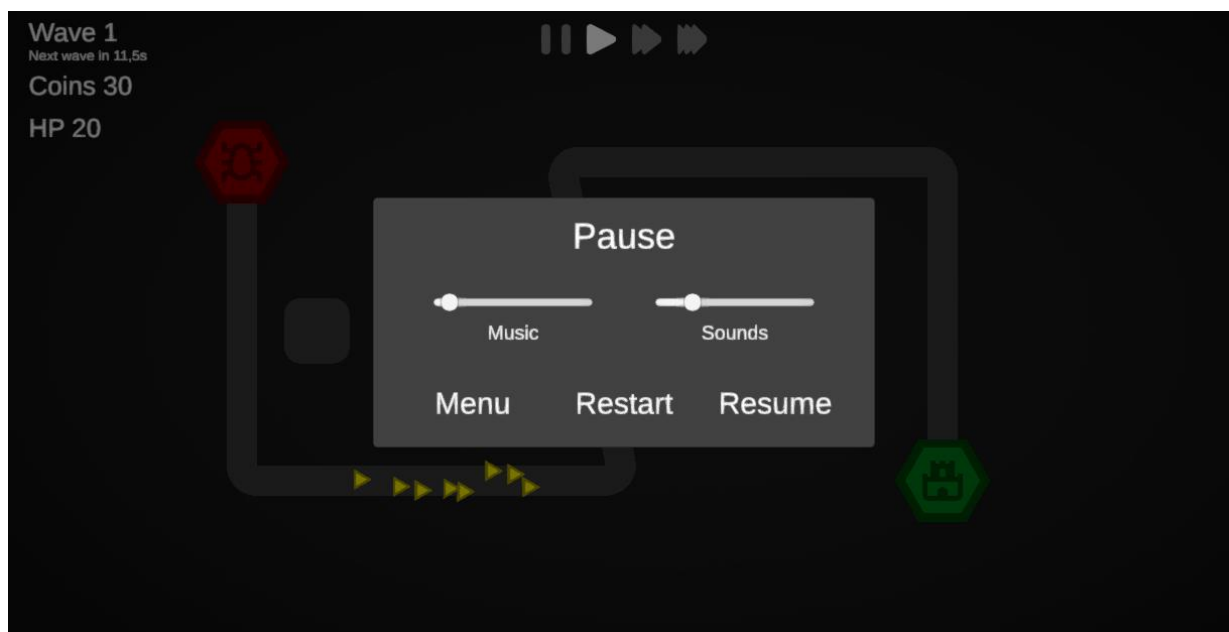


Рисунок 3.8 – Меню паузы [разработано автором]

Наконец, был создан класс EnemySpawn, который создает врагов случайного типа раз в единицу времени. С течением времени количество

врагов, их сила и награда за победу волны постепенно увеличиваются. Реализация класса представлена в листинге Б.2.

3.1.3 Разработка системы сохранений

Игра сохраняет данные об рекордах и настройках в файле сохранения в формате JSON. Для описания структуры используется класс `DataStruct`, реализация которого представлена в листинге 3.2.

Листинг 3.2 – Класс `DataStruct` [разработано автором]

```
public class DataStruct
{
    public float musicVolume;
    public float soundVolume;
    public Dictionary<string, int> levelRecords;
}
```

Сам файл сохранения в формате JSON представлен в листинге 3.3.

Листинг 3.3 – Файл сохранения [разработано автором]

```
{
  "musicVolume": 0.0250225775,
  "soundVolume": 0.1999524,
  "levelRecords": {
    "Level 1": 85,
    "Level 2": 15
  }
}
```

Для изменения самого файла сохранения используется класс `SaveFileHandler`, который имеет прямые методы загрузки и выгрузки файла. Методы бизнес-логики описаны в классе `SaveManager`. Данный класс использует в себе методы класса `SaveFileHandler` для непосредственной работы с файлом сохранения. Классы `SaveFileHandler` и `SaveManager` представлены в листингах Б.3 и Б.4 соответственно.

3.1.4 Разработка интерфейсных сцен

Помимо игровых уровней в игре присутствуют две интерфейсные сцены [21]: стартовая сцена и сцена меню.

Стартовая сцена необходима для создания элементов звукового сопровождения (так как они не уничтожаются между сценами) и для проверки состояния файла сохранения. Фоном стартовой сцены выступает фрагмент игрового процесса, на котором размещены несколько башен, а враги

периодически появляются на экране. Разработанная сцена представлена на рисунке 3.9.

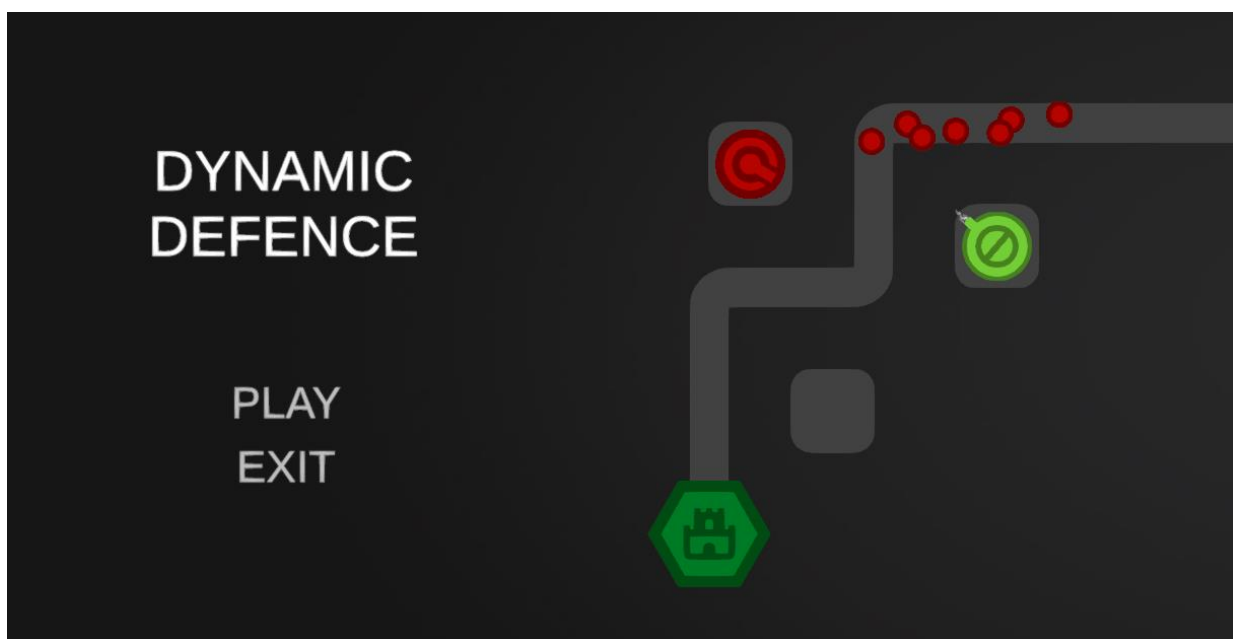


Рисунок 3.9 – Стартовая сцена [разработано автором]

Сцена меню представляет собой меню, в котором пользователь может выбрать уровень и ознакомиться со своими рекордами. Сцена с меню представлена на рисунке 3.10.

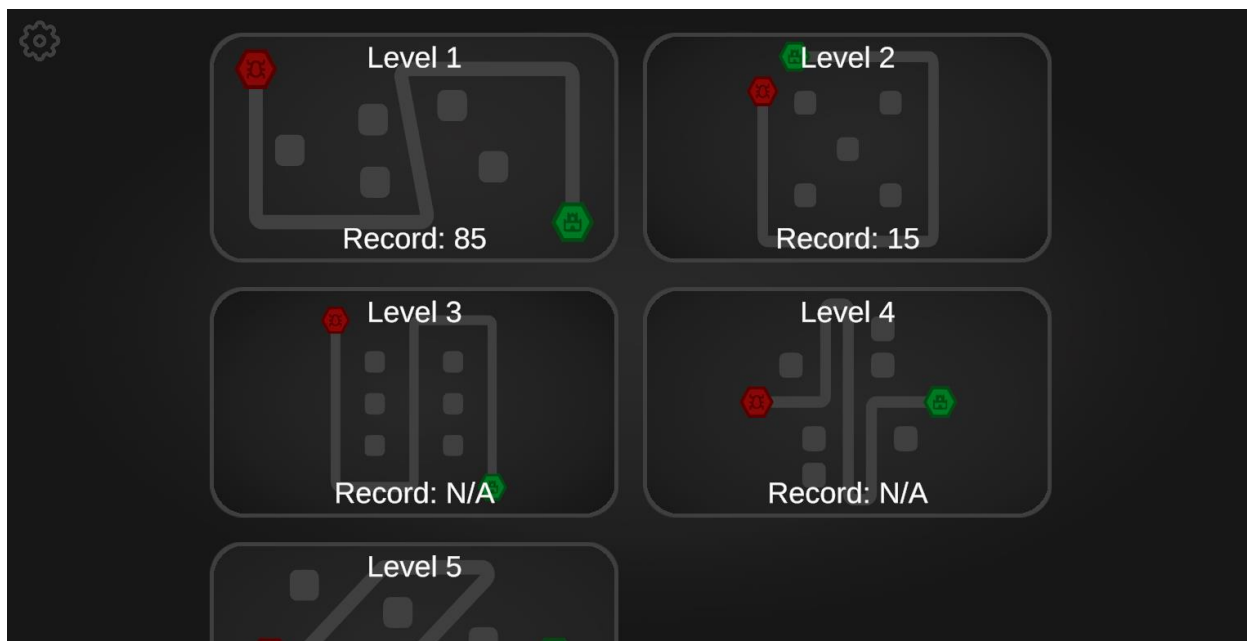


Рисунок 3.10 – Сцена меню [разработано автором]

В итоге со стартовой сцены пользователь попадает в сцену меню, а из сцены меню уже в сцену игрового уровня и обратно. Граф переходов между сценами представлен на рисунке 3.11.

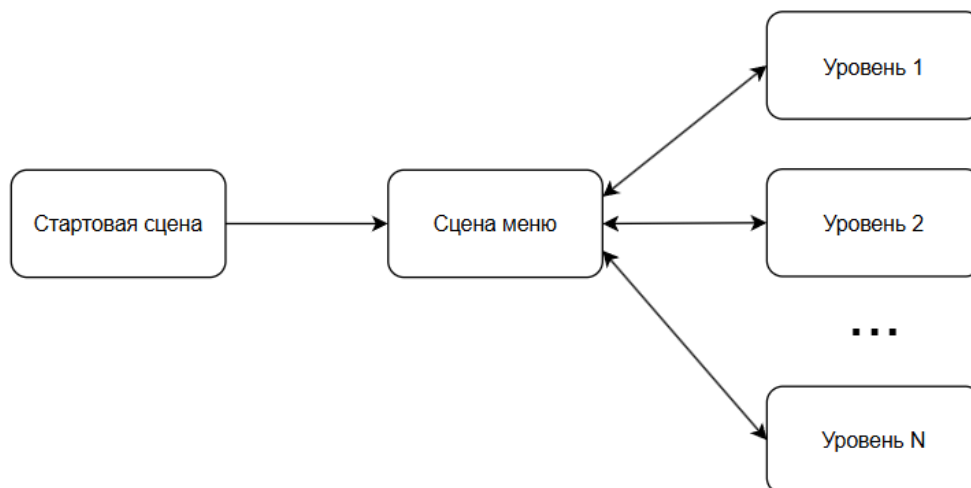


Рисунок 3.11 – Граф переходов между сценами [разработано автором]

3.2 Разработка модуля динамической корректировки сложности

Разработаем модуль динамической корректировки сложности в виде класса MLDDA, наследуемого от класса Agent [22]. Модель ИИ постоянно будет принимать параметры игровой среды, и по завершении каждой волны будет выдавать несколько выходов, исходя из которых будут изменяться параметры следующей волны. Обозначим эти выходы:

- Изменение количества врагов (целое число от -1 до 1).
- Изменение награды за успешное завершение волны (целое число от -5 до 5).
- Изменение множителя силы врагов (число от -0.2 до 0.2).

Переопределим метод OnActionReceived, который вызывается каждый раз при получении результата работы модели ИИ. Реализация представлена в листинге 3.4.

Листинг 3.4 – Метод OnActionReceived класса MLDDA [разработано автором]

```

public override void OnActionReceived(ActionBuffers actions)
{
    if (isChange)
    {
        int enemyCountDelta = actions.DiscreteActions[0] - 1;
        int waveAwardDelta = actions.DiscreteActions[1] - 5;
        float wavePowerDelta
=actions.ContinuousActions[0]*0.5f;
        enemySpawn.setDelta(enemyCountDelta, waveAwardDelta,
wavePowerDelta);
    }
}

```

Обозначим также параметры, которые будет принимать модель:

- Длина пути.
- Текущие значения изменяемых параметров (количество врагов, награда за успешное завершение волны и сила врагов).
- Тип врагов (за последние n волн).
- Дистанция от начала уровня до дальней точки сосредоточения башен (за последние n волн).
- Дистанция, пройденная последним врагом (за последние n волн).

Дальняя точка сосредоточения башен считается как среднее между критическими точками (то есть точками, по которой башня еще может стрелять) каждой из башен с учетом весов в зависимости от их уровней улучшения.

Переопределим метод `CollectObservations`, который будет собирать данные об уровне. При отсутствии данных об предыдущих волнах будет записываться число -1. Реализация представлена в листинге 3.5.

Листинг 3.5 – Метод `CollectObservations` класса `MLDDA` [разработано автором]

```
public override void CollectObservations(VectorSensor sensor)
{
    sensor.AddObservation(isChange);
    sensor.AddObservation(pathDistance);
    sensor.AddObservation(enemySpawn.getEnemyCount());
    sensor.AddObservation(enemySpawn.getWaveAward());
    sensor.AddObservation(enemySpawn.getWavePower());
    for (int i = 0; i < numObservations; i++) {
        if (waveLiveTimeList.Count > i) {
            sensor.AddObservation(waveEnemyTypeList[i]);
            sensor.AddObservation(lastDistanceList[i]);
            sensor.AddObservation(lastHitDistanceList[i]);
        }
        else
        {
            for (int j = 0; j < 3; j++)
            {
                sensor.AddObservation(-1);
            }
        }
    }
}
```

Обозначим также условие удовлетворенности игрока: если дистанция, пройденная последним врагом равна дистанции до точки сосредоточения башен, то игрок удовлетворен. В связи с этим будем выдавать агенту положительную награду, если это условие выполняется, и отрицательную, если нет.

Реализуем метод `endWave`, который будет вызываться каждый раз при окончании волны. В нем будут собираться новые параметры об уровне, а также выдаваться награда. Реализация представлена в листинге 3.6.

Листинг 3.6 – Метод `endWave` класса `MLDDA` [разработано автором]

```
public void endWave(bool b)
{
    addToList(lastDistanceList, towersLastDistance());
    if (lastHitPositions.Count == 0)
    {
        lastHitPositions.Add(line.GetPosition(line.positionCount - 1));
    }
    if (!b) addToList(lastHitDistanceList, pathDistance);
    else addToList(lastHitDistanceList,
        calculatePathDistance(lastHitPositions[lastHitPositions.Count - 1]));

    float ld = lastDistanceList[lastDistanceList.Count - 1];
    float lhd = lastHitDistanceList[lastHitDistanceList.Count - 1];

    if (!b) SetReward(-5f);
    SetReward((-Mathf.Abs(ld - lhd) / pathDistance + 0.3f) * 10);

    lastHitPositions = new List<Vector3>();
    isChange = true;
}
```

Полный код класса `MLDDA` представлен в листинге В.1.

В Unity создадим пустой объект и прикрепим к нему написанный скрипт и компонент `Behaviour Parameters`, который необходим для корректного обучения и работы модели. В этом компоненте необходимо верно указать размер входа модели, который в случае наблюдения за последними 5 волнами равен 20, а также тип и размеры выходов. На рисунке 3.12 представлена настройка созданного объекта.

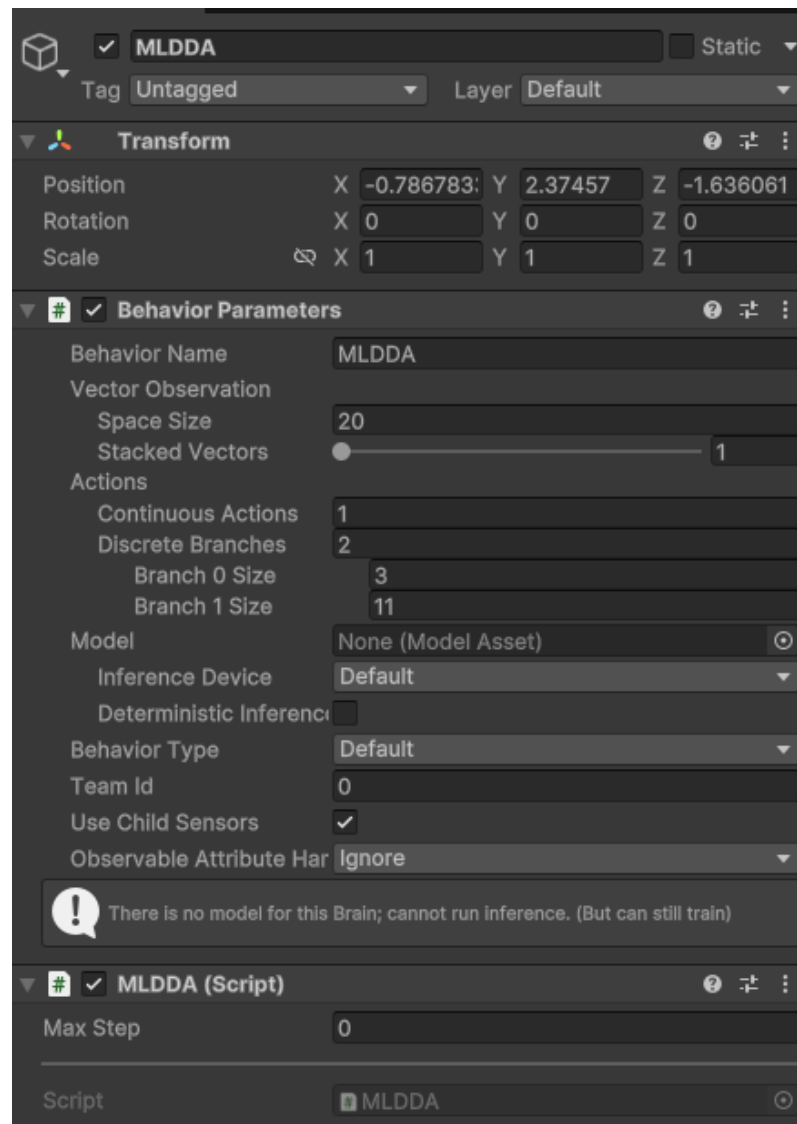


Рисунок 3.12 – Настройка объекта модели ИИ для DDA [разработано автором]

3.3 Интеграция модуля в игровой процесс

3.3.1 Создание и обучение модели симуляции игрока

Поскольку необходимы данные для обучения модели ИИ для DDA, необходимо реализовать вторую модель ИИ, симулирующая поведение игрока. Принцип ее написания не отличается от первой модели. Агент получает награду в случае успешного прохождения волны и штраф в случае неуспешного.

Обучим полученную модель с помощью утилиты `mlagents-learn` [23], и в дальнейшем будем использовать ее для обучения модели ИИ для DDA.

3.3.2 Обучение модели модуля динамической корректировки сложности

Создадим файл конфигурации, в котором обозначаются параметры обучения и структурные параметры модели. Часть данного файла представлена в листинге 3.7.

Листинге 3.7 – Конфигурационный файл [разработано автором]

```
behaviors:
  MLDDA:
    trainer_type: ppo
    hyperparameters:
      batch_size: 4096
      buffer_size: 40960
      learning_rate: 0.0005
      beta: 0.002
      epsilon: 0.2
      lambda: 0.95
      num_epoch: 5
      shared_critic: false
      learning_rate_schedule: linear
      beta_schedule: linear
      epsilon_schedule: linear
    checkpoint_interval: 100000
    network_settings:
      normalize: true
      hidden_units: 256
      num_layers: 2
      vis_encode_type: simple
      memory: null
      goal_conditioning_type: hyper
      deterministic: false
```

Запустим обучение модели с помощью утилиты `mlagent-learn`, и после этого запустим сцену. На рисунке 3.13 представлены результаты обучения, предоставленные с помощью Tensorboard [24]. Как видно из рисунка, средняя награда стабильно положительная и возрастает с ходом обучения, а энтропия модели падает.

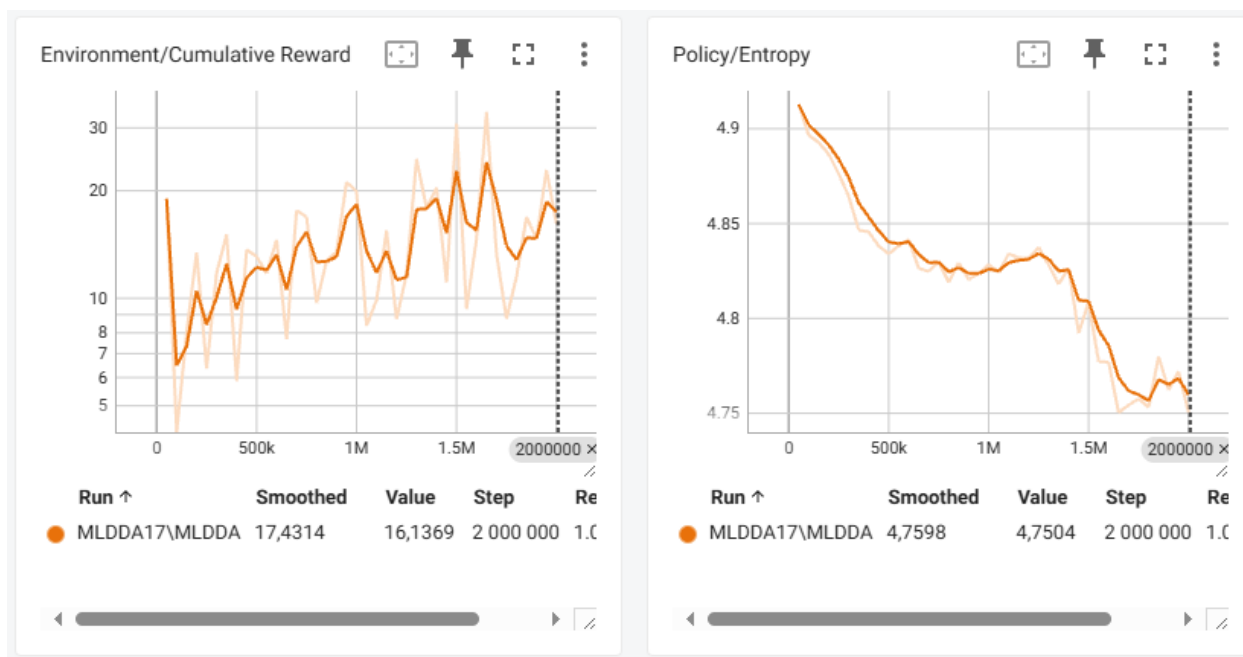


Рисунок 3.13 – Результаты обучения модели [разработано автором]

3.4 Тестирование игрового продукта

Игровой продукт был протестирован на функциональность и производительность. Тестирование показало, что весь разработанный функционал работает корректно. Производительность игры осталась стабильной на протяжении всех фаз тестирования, серьезных сбоев и зависаний не было.

Отдельно была протестирована работа модуля динамической корректировки сложности. Для этого были просмотрены логи игры (рисунок 3.14). Как видно из логов игры, награда, получаемая агентом, увеличивается. Это значит модель обучилась корректно и работает так, как было задумано.

```
[17:24:02] 0.2860595
UnityEngine.Debug:Log (object)

[17:24:02] -1 -4 0,2816423
UnityEngine.Debug:Log (object)      Reward: 32,81021

[17:24:04] 0.2860595
UnityEngine.Debug:Log (object)

[17:24:04] 1 -5 0,2215482
UnityEngine.Debug:Log (object)      Reward: 35,6708

[17:24:07] 0.2566294
UnityEngine.Debug:Log (object)

[17:24:07] 0 -1 -0,08423698
UnityEngine.Debug:Log (object)      Reward: 38,2371
```

Рисунок 3.14 – Логи игры [разработано автором]

3.5 Вывод к разделу 3

Была реализован игровой продукт, а именно было разработано графическое и звуковое оформление, логика игры, а также модуль динамической корректировки сложности. Игра была успешно протестирована на функциональность и производительность.

4 Экономический раздел

4.1 Организация и планирование работ по теме

В ходе реализации ВКР осуществляются проектирование и разработка компьютерного приложения – игры в жанре «Tower Defence» с применением искусственного интеллекта для динамической корректировки сложности.

В составе работы задействовано 3 человека:

- Руководитель ВКР (Алпатов Алексей Николаевич, кандидат технических наук, доцент, доцент кафедры ИИ) – отвечает за грамотную постановку задачи, контролирует отдельные этапы работы, вносит необходимые коррективы и оценивает выполненную работу в целом.

- Консультант (консультант по экономической части выпускной квалификационной работы, Жемерикин Олег Игоревич, кандидат экономических наук, доцент кафедры экономики) – отвечает за консультирование в области экономической части выпускной квалификационной работы.

- Разработчик (студент 4-го курса, Ахмин Максим Сергеевич, ИКБО-03-21) – реализация всех поставленных задач, в том числе проектирование, разработка и тестирование продукта и подготовка проектной документации.

Состав задействованных в работе участников представлен на рисунке 4.1.

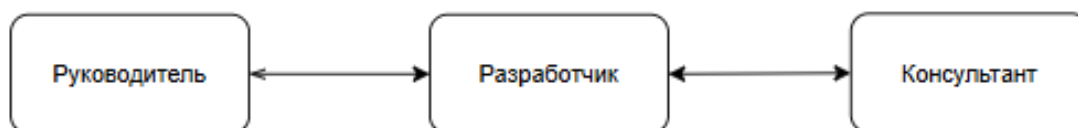


Рисунок 4.1 – Состав участников [разработано автором]

4.1.1 Организация работ

Организация работ осуществляется на базе календарного планирования, которое включает в себя:

- Планирование содержания проекта и декомпозицию работ.
- Определение последовательности работ.

- Планирование сроков, длительностей и логических связей работ.
- Построение диаграммы Ганта [25].

В таблице 4.1 приведён состав этапов работы с указанием длительности и ответственных за выполнение.

Таблица 4.1 – Этапы разработки [разработано автором]

№	Название этапа	Исполнитель	Трудоемкость, чел/дни	Продолжительность работ, дни
1	Формирование требований к проекту			4
1.1	Обследование объекта и обоснование необходимости создания проекта	Руководитель ВКР	1	
		Разработчик	2	
1.2	Формирование требований пользователя к проекту	Руководитель ВКР	1	
		Разработчик	1	
1.3	Оформление отчета о выполненной работе	Разработчик	1	
2	Разработка концепции проекта			6
2.1	Изучение объекта	Руководитель ВКР	1	
		Разработчик	2	
2.2	Разработка вариантов реализации, выбор и согласование варианта, удовлетворяющего требованиям	Руководитель ВКР	2	
		Разработчик	2	
2.3	Разработка игрового концепт документа	Разработчик	1	
2.4	Оформление отчета о выполненной работе	Разработчик	1	
3	Техническое задание			4
3.1	Разработка и утверждение технического задания на игровой продукт	Руководитель ВКР	2	
		Разработчик	2	
3.2	Разработка документации	Разработчик	2	
4	Эскизный (пилотный проект)			17
4.1	Реализация основных игровых механик	Разработчик	7	
4.2	Реализация игрового уровня	Разработчик	5	
4.3	Разработка и реализация предварительного пользовательского интерфейса	Разработчик	3	
4.4	Разработка документации на игровой продукт	Разработчик	2	
5	Технический проект			29
5.1	Реализация побочных игровых механик	Разработчик	7	

Продолжение таблицы 4.1

5.2	Реализация и внедрение модели ИИ	Разработчик	7	
5.3	Реализация графического оформления игры	Разработчик	5	
5.4	Реализация звукового сопровождения игры	Разработчик	4	
5.5	Улучшение пользовательского интерфейса	Разработчик	2	
5.6	Локальное тестирование игры на предмет ошибок и багов	Разработчик	2	
5.7	Разработка документации на игровой продукт	Разработчик	2	
6	Рабочая документация			7
6.1	Разработка рабочей документации на игровой продукт	Разработчик	3	
6.2	Формирование комплекта рабочей документации на игровой продукт	Разработчик	2	
6.3	Расчёт затрат на проведение работ	Разработчик	2	
		Консультант по экономической части	1	
7	Ввод в действие			5
7.1	Сборка проекта	Разработчик	2	
7.2	Тестирование продукта	Разработчик	3	
Итого				72

Из данной таблицы видно, что общий срок разработки займет 72 дня.

В таблице 4.2 приведён календарный план выполнения этапов проектирования и разработки компьютерного приложения, планирование выполняется с учётом шестидневной рабочей недели, выходных и праздничных дней [26].

Первым днём планирования является первый день восьмого семестра.

Таблица 4.2 – Календарный план выполнения проекта [разработано автором]

Этап	Дата начала	Дата окончания	Количество рабочих дней	Исполнители
1 Формирование требований к проекту				
1.1 Обследование объекта и обоснование необходимости создания проекта	10.02.2025	11.02.2025	2	Руководитель ВКР Разработчик

Продолжение таблицы 4.2

1.2 Формирование требований пользователя к проекту	12.02.2025	12.02.2025	1	Руководитель ВКР Разработчик
1.3 Оформление отчета о выполненной работе	13.02.2025	13.02.2025	1	Разработчик
2. Разработка концепции проекта				
2.1 Изучение объекта	14.02.2025	15.02.2025	2	Руководитель ВКР Разработчик
2.2 Разработка вариантов реализации, выбор и согласование варианта, удовлетворяющего требованиям	17.02.2025	18.02.2025	2	Руководитель ВКР Разработчик
2.3 Разработка игрового концепт документа	19.02.2025	19.02.2025	1	Разработчик
2.4 Оформление отчета о выполненной работе	20.02.2025	20.02.2025	1	Разработчик
3. Техническое задание				
3.1 Разработка и утверждение технического задания на игровой продукт	21.02.2025	22.02.2025	2	Руководитель ВКР Разработчик
3.2. Разработка документации	24.02.2025	25.02.2025	2	Разработчик
4. Эскизный (пилотный) проект				
4.1 Реализация основных игровых механик	26.02.2025	05.03.2025	7	Разработчик
4.2 Реализация игрового уровня	06.03.2025	12.03.2025	5	Разработчик
4.3 Разработка и реализация предварительного пользовательского интерфейса	13.03.2025	15.03.2025	3	Разработчик
4.4 Разработка документации на игровой продукт	17.03.2025	18.03.2025	2	Разработчик
5. Технический проект				
5.1 Реализация побочных игровых механик	19.03.2025	26.03.2025	7	Разработчик
5.2 Реализация и внедрение модели ИИ	27.03.2025	03.04.2025	7	Разработчик
5.3 Реализация графического оформления игры	04.04.2025	09.04.2025	5	Разработчик
5.4 Реализация звукового сопровождения игры	10.04.2025	14.04.2025	4	Разработчик

Продолжение таблицы 4.2

5.5	Улучшение пользовательского интерфейса	15.04.2025	16.04.2025	2	Разработчик
5.6	Локальное тестирование игры на предмет ошибок и багов	17.04.2025	18.04.2025	2	Разработчик
5.7	Разработка документации на игровой продукт	19.04.2025	21.04.2025	2	Разработчик
6. Рабочая документация					
6.1	Разработка рабочей документации на игровой продукт	22.04.2025	24.04.2025	3	Разработчик
6.2	Формирование комплекта рабочей документации на игровой продукт	25.04.2025	26.04.2025	2	Разработчик
6.3	Расчёт затрат на проведение работ	28.04.2025	29.04.2025	2	Разработчик Консультант по экономической части
7. Ввод в действие					
7.1	Сборка проекта	30.04.2025	02.05.2025	2	Разработчик
7.2	Тестирование продукта	03.05.2025	06.05.2025	3	Разработчик

4.1.2 График проведения работ

Диаграмма Ганта широко используется для визуализации хода выполнения задач, планирования ресурсов, графика рабочего времени и других данных, которые представляются набором временных интервалов. Она необходима для соблюдения сроков выполнения работ, и, соответственно, анализа хода проектирования и/или разработки ИТ-решения в рамках ВКР.

Календарный график исполнения работы с учётом производственного календаря [26] и разработанный в соответствии с таблицей 4.2, представлен на рисунке Г.1. Из рисунка Г.1 так же видно, что общий срок разработки составит 72 дня.

4.2 Расчет стоимости проведения работ

Себестоимость работ (проектирования и/или разработки ИТ-решения) складывается из затрат по следующим статьям:

- Статья «Сырье и материалы» + ТЗР (15%) от Σ итого по материалам.
- Статья «Основная заработная плата».
- Статья «Дополнительная заработная плата» – 20-30% от основной заработной платы.
- Статья «Страховые взносы» – 30% от ФОТ, а также ставка за травматизм.
- Статья «Амортизация».
- Статья «Прочие расходы».

В выпускной квалификационной работе объем затрат на НИР и ОКР (проектирования и/или разработки ИТ-решения) был проведен методом калькулирования.

4.2.1 Статья «Сырье и материалы»

К статье «Сырье и материалы» относится: стоимость материалов, покупных изделий, полуфабрикатов, комплектующих изделий и других материальных ценностей, расходуемых непосредственно в процессе выполнения работ (проектирования и/или разработки ИТ-решения). Потребность в материальных ресурсах определяется на основе материальных спецификаций, в которых указывается наименование, количество и цена используемых в процессе работы материальных ценностей.

При описании стоимости расходов на «Сырье и материалы» будем считать, что предприятие (университет) в своей учетной политике придерживается лимита стоимости основных средств в 100 тыс. руб. А это значит, что имущество со сроком полезного использования более 12 месяцев, но стоимостью ниже установленного лимита можно относить к расходам [27, 28].

В стоимость материальных затрат включаются транспортно-заготовительные расходы, которые возьмем на уровне 15 % стоимости затрат

по статье. В эту статью включаются также затраты на оформление комплекта документов. Расчет стоимости материалов представлен в таблице 4.3.

Таблица 4.3 – Стоимость материалов [разработано автором]

№ п/п	Наименование материалов	Единицы измерения	Количество	Цена за единицу (руб)	Стоимость (руб)
1	Флешка 2Гб	шт	1	550	550
2	Бумага А4	пачка	1	250	250
3	Картридж для принтера	шт	1	3350	3350
4	Ручка шариковая	шт	2	30	60
Итого материалов					4 210,0
Транспортно-заготовительные расходы					631,5
Итого					4 841,5

Таким образом общая сумма затрат по статье «Сырье и материалы» составит 4 841,5 руб.

4.2.2 Статья «Основная заработная плата»

К статье «Основная заработная плата» относится оплата труда научных работников, инженерно-технических работников и рабочих, непосредственно занятых выполнением конкретной работы, а также заработная плата работников внештатного состава, привлекаемых к ее выполнению. Расчет заработной платы проводится исходя из численности различных категорий исполнителей, трудоемкости отдельных видов выполняемых ими работ и их тарифной ставки.

Согласно Производственному календарю на 2025 год, годовой фонд рабочего времени при *шестидневной рабочей неделе* составит 299 рабочих дней.

Следовательно, дневная тарифная ставка (ТС) для месячного оклада по каждому работнику может быть рассчитана по формуле (4.1).

$$ТС = \frac{ОК \times 12}{НРВ} = \frac{ОК \times 12}{299} \text{ руб./д.} \quad (4.1)$$

где

ОК – месячный оклад участника проекта;

12 – число месяцев в году;

НРВ – годовой фонд рабочего времени при шестидневной рабочей неделе.

В таблице 4.4 приведен расчет заработной платы исполнителей. Так как руководитель и консультант - преподаватели университета, то расчет дневной тарифной ставки проводим по формуле (4.1). Количество рабочих дней для каждой роли определяется на основе таблице 4.1 с учетом участия в выполнении работ.

Таблица 4.4 – Расчет основной заработной платы [разработано автором]

№ п/п	Наименование этапа	Исполнитель (должность)	Мес. оклад (руб)	Трудоемкость (чел/дни)	Оплата за день (руб)	Оплата за этап (руб)
1	Формирование требований к проекту	Руководитель	130 000	2	5 217,4	10 434,8
		Разработчик	60 000	4	2 408,0	9 632,1
2	Разработка концепции проекта	Руководитель	130 000	3	5 217,4	15 652,2
		Разработчик	60 000	6	2 408,0	14 448,2
3	Техническое задание	Руководитель	130 000	2	5 217,4	10 434,8
		Разработчик	60 000	4	2 408,0	9 632,1
4	Эскизный (пилотный) проект	Разработчик	60 000	17	2 408,0	40 936,5
5	Технический проект	Разработчик	60 000	29	2 408,0	69 832,8
6	Рабочая документация	Консультант по экономической части	105 000	1	4 214,0	4 214,0
		Разработчик	60 000	7	2 408,0	16 856,2
7	Ввод в действие	Разработчик	60 000	5	2 408,0	12 040,1
Итого						214 113,7

Таким образом общая сумма затрат по статье «Основная заработная плата» составит 214 113,7 руб.

4.2.3 Статья «Дополнительная заработная плата»

На эту статью относятся выплаты, предусмотренные законодательством о труде за неотработанное по уважительным причинам время; оплата очередных и дополнительных отпусков; времени, связанного с выполнением государственных и общественных обязанностей; выплата вознаграждения за выслугу лет и т.п.

В среднем расходы дополнительной заработной платы (ДЗП) составляют 20-30% от суммы основной заработной платы и могут быть рассчитаны по формуле (4.2).

$$\text{ДЗП} = (20 \dots 30)\% \times \text{ОЗП} \quad (4.2)$$

где

ОЗП – основная заработная плата.

Например, если расходы на дополнительную заработную плату принять в размере 20%, то дополнительная заработная плата по рассматриваемому проекту составит 42 822,7 руб.

$$\text{ДЗП} = 0,2 \times 214\,113,7 = 42\,822,7 \text{ руб.}$$

В процессе определения сметы затрат вводится понятие «фонд оплаты труда» (ФОТ), представляющий собой сумму основной (ОЗП) и дополнительной заработной платы (ДЗП). Фонд оплаты труда используется при расчете взносов в социальный фонд. Фонд оплаты труда может быть рассчитан по формуле (4.3).

$$\text{ФОТ} = \text{ОЗП} + \text{ДЗП} \quad (4.3)$$

Для данного проекта фонд оплаты труда составит 256 936,4 руб.

$$\text{ФОТ} = 214\,113,7 + 42\,822,7 = 256\,936,4 \text{ руб.}$$

Таким образом общая сумма затрат по статье «Дополнительная заработная плата» составит 42 822,7 руб.

4.2.4 Статья «Страховые взносы»

В соответствии со статьей 425 НК РФ в 2025 году для лиц, которые производят выплаты и вознаграждения физическим лицам (за исключением плательщиков, для которых установлены пониженные тарифы страховых взносов), установлен Единый тариф страховых взносов 30% [29] по направлениям:

- Обязательное пенсионное страхование (ОПС).
- Обязательное социальное страхование на случай временной нетрудоспособности и в связи с материнством (ОСС).
- Обязательное медицинское страхование (ОМС).

Также в расчёте необходимо учесть ставку взносов на травматизм, которая может составлять от 0,2% до 8,5%. Для РТУ МИРЭА данная ставка составляет 0,2% [30]. Следовательно, страховые взносы (СВ) могут быть рассчитаны по формуле (4.4).

$$СВ = 30,2\% \times \text{ФОТ} \quad (4.4)$$

Таким образом общая сумма затрат по статье «Страховые взносы» составит 77 594,8 руб.

$$СВ = 0,302 \times 256\,936,4 = 77\,594,8 \text{ руб.}$$

4.2.5 Статья «Амортизация»

Основные средства в структуре себестоимости учитываются только через амортизацию. Т.е. затраты на закупку оборудования или оборудование, на котором выполнялась научная работа (проект, выпускная квалификационная работа), учитываются через определение амортизационных отчислений.

Амортизация — отчисления части стоимости основных фондов (например, стоимости оборудования) для возмещения их износа. Амортизация включается в издержки производства. Размер амортизационных отчислений определяется на основе установленных норм и балансовой стоимости основных фондов, на которые начисляется амортизация.

Будем считать, что предприятие (университет) в своей учетной политике придерживается лимита стоимости основных средств в 100 тыс. руб. А это значит, что объекты стоимостью до этого предела не относятся к амортизируемому имуществу [28].

Существуют несколько способов начисления амортизации. Способ начисления, который выбирает предприятие, обязательно указывается в учетной политике организации.

В рамках определения стоимости работ по проекту предлагается использовать линейный способ вычисления амортизационных отчислений.

К амортизационным отчислениям во время разработки проекта будет относиться сумма используемого во время разработки проекта оборудования к отношению времени использованного данного оборудования.

Амортизационные отчисления могут быть рассчитаны по формуле (4.5).

$$A = \frac{C}{T} \quad (4.5)$$

где

A — месячная сумма амортизационных отчислений,

C — первоначальная стоимость объекта,

T — срок полезного использования в месяцах.

В таблице 4.5 представлен пример расчетов амортизационных отчислений по использованному во время разработки проекта оборудованию.

Таблица 4.5 – Амортизационные отчисления [разработано автором]

Наименование оборудования	Первоначальная стоимость объекта, руб.	Срок полезного использования, мес.	Месячная сумма амортизационных отчислений	Период эксплуатации в месяцах	Сумма, руб.
Компьютер	120 000	36	3 333,3	4	13 333,3
Итого					13 333,3

Таким образом общая сумма затрат по статье «Амортизация» составит 13 333,3 руб.

4.2.6 Статья «Прочие расходы»

К прочим расходам прежде всего отнесены накладные расходы. Это расходы на содержание и ремонт зданий, сооружений, оборудования, инвентаря. Это затраты, сопутствующие основному производству, но не связанные с ним напрямую, не входящие в стоимость труда и материалов.

В рамках реализации проекта это могут быть расходы, связанные с эксплуатацией, содержанием, и арендой площадей, с оплатой консультационных, информационных и клиринговых услуг, а также с командировочными расходами.

Сумма прочих расходов определяется процентом от суммы основной заработной платы и на разных предприятиях, в зависимости от их структуры, технологического процесса и системы управления находится в широком диапазоне — от 100 до 130%. Прочие расходы (ПР) могут быть рассчитаны по формуле (4.6)

$$\text{ПР} = (100 \dots 130)\% \times \text{ОЗП} \quad (4.6)$$

Взяв в качестве процента значение 110% в результате имеем общую сумму затрат по статье «Амортизация» в 225 525,08 руб.,

$$\text{ПР} = 1,10 \times 214\,113,7 = 235\,525,08 \text{ руб.}$$

4.2.7 Калькуляция затрат по проекту

Вышеперечисленные статьи затрат и результаты расчётов по ним обобщаются в таблице 4.6.

Таблица 4.6 – Полная себестоимость работ [разработано автором]

№ п/п	Номенклатура статей расходов	Затраты (руб.)	Доля затрат, %
1	Сырье и материалы	4 841,50	0,8%
2	Основная заработная плата	214 113,71	36,4%
3	Дополнительная заработная плата	42 822,74	7,3%
4	Страховые взносы	77 594,81	13,2%
5	Амортизация	13 333,30	2,3%
6	Прочие расходы	235 525,08	40,0%
Итого		588 231,15	100,0%

Для визуализации долевого состава статей затрат в общей себестоимости представим круговую диаграмму на рисунке 4.2.

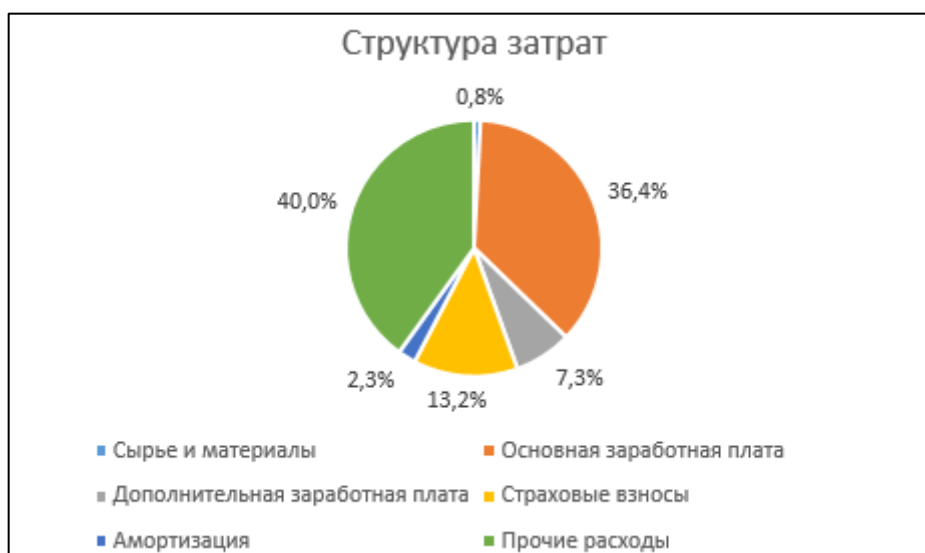


Рисунок 4.2 – Структура затрат [разработано автором]

Так как предполагается, что разработанный продукт в дальнейшем будет реализован, рассчитаем договорную цену (ДЦ) по формуле (4.7).

$$\text{ДЦ} = \text{С} + \text{П} + \text{НДС} \quad (4.7)$$

где

С – себестоимость разработанного продукта,

П – прибыль разработанного продукта,

НДС – налог на добавленную стоимость.

Норма прибыли составляет 10-30% от стоимости разработки.

Взяв в качестве процента значение 20% получим, что прибыль будет равна 117 646,23 руб.

$$\Pi = 588\,231,15 \times 20\% = 117\,646,23 \text{ руб.}$$

Налог на добавленную стоимость может быть рассчитан по формуле (4.8).

$$\text{НДС} = (C + \Pi) \times 20\% \quad (4.8)$$

В результате получаем, что налог на добавленную стоимость равен 141 175,28 руб.

$$\text{НДС} = (588\,231,15 + 117\,646,23) \times 20\% = 141\,175,48 \text{ руб.}$$

Таким образом, договорная цена будет равна 847 052,85 руб.

$$\text{ДЦ} = C + \Pi + \text{НДС} = 847\,052,85 \text{ руб.}$$

4.3 Вывод к разделу 4

В рамках данного экономического раздела было выполнено планирование работ по теме, был проведен расчет стоимости затрат и определена договорная цена. На основе полученных значений можно сделать вывод о целесообразности разработки данного проекта.

ЗАКЛЮЧЕНИЕ

В ходе выпускной квалификационной работы были успешно выполнены все поставленные задачи, а поставленная цель была достигнута. Была спроектирована и реализована компьютерная игра в жанре «Tower Defence» с модулем динамической корректировки сложности, основанным на технологиях искусственного интеллекта.

На этапе исследовательской работы был проведен анализ существующих игр жанра «Tower Defence», изучены их механики, способы балансировки и адаптации сложности. Это позволило определить ключевые принципы жанра и выявить необходимость использования ИИ для создания гибкой системы сложности.

На этапе проектирования был выбран стек технологий, а также разработаны гейм-дизайн документ, архитектуры игрового продукта и модуля динамической корректировки сложности. В качестве игрового движка был выбран Unity, а для реализации модуля ИИ – Python и ML-Agents.

На технологическом этапе было разработано графическое и звуковое оформление игры, были реализованы игровые и интерфейсные сцены, был описан, реализован и интегрирован модуль динамической корректировки сложности. Разработанный продукт был успешно протестирован на функциональность и производительность.

На экономическом этапе была описана организация выполнения работ, а также рассчитана стоимость ее выполнения. Был сделан вывод о целесообразности разработки данного проекта.

Основные положения и результаты исследования были апробированы в форме публикации статьи [31].

During the final qualifying work, all the tasks were successfully completed, and the goal was achieved. A computer game in the "Tower Defense" genre with a dynamic difficulty adjustment module based on artificial intelligence technologies was designed and implemented.

At the research stage, an analysis of existing games of the "Tower Defense" genre was carried out, their mechanics, ways of balancing and adapting complexity were studied. This allowed us to identify the key principles of the genre and identify the need to use AI to create a flexible complexity system.

At the design stage, a technology stack was selected, and a game design document, game product architectures, and a dynamic complexity adjustment module were developed. Unity was chosen as the game engine, and Python and ML-Agents were chosen to implement the AI module.

At the technological stage, the graphic and sound design of the game was developed, game and interface scenes were implemented, and a dynamic difficulty adjustment module was described, implemented, and integrated. The developed product has been successfully tested for functionality and performance.

At the economic stage, the organization of the work was described, as well as the cost of its implementation was calculated. It was concluded that it was advisable to develop this project.

The main results of the research were tested in the article publication [31].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Игровая индустрия за 2024 год в цифрах: самые продаваемые игры и обсуждаемые издатели [Электронный ресурс]. – URL: <https://dtf.ru/gameindustry/3312702-igrovaya-industriya-za-2024-god-v-cifrah-samy-prodavaemye-igry-i-obsuzhdaemye-izdateli> (дата обращения 10.03.2025).
2. Strojny A., Strojny P., Wójcik A., Krzywicka K. Player involvement as a result of difficulty: An introductory study to test the suitability of the motivational intensity approach to video game research // PLoS ONE. 2023. – Vol. 18, No. 3. – DOI: 10.1371/journal.pone.0281880.
3. Vang C. The Impact of Dynamic Difficulty Adjustment on Player Experience in Video Games // University of Minnesota, 2022. – URL: <https://digitalcommons.morris.umn.edu/horizons/vol9/iss1/7/> (дата обращения 11.03.2025). – Текст: электронный.
4. ГОСТ 7.32-2017. Межгосударственный стандарт. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления": введен в действие Приказом Росстандарта от 24.10.2017 N 1494-ст: дата введения 2018-01-07. – URL: https://www.consultant.ru/document/cons_doc_LAW_292293/ (дата обращения: 10.05.2025). – Текст: электронный.
5. ГОСТ Р 7.0.100-2018. Национальный стандарт Российской Федерации. Библиографическая запись. Библиографическое описание: общие требования и правила составления: Приказом Федерального агентства по техническому регулированию и метрологии (Росстандартом) от 03 декабря 2018 N 1050-ст: дата введения 2019-01-07. – URL: <https://docs.cntd.ru/document/1200161674> (дата обращения: 10.05.2025). – Текст: электронный.
6. СМКО МИРЭА 7.5.1/03.П.67-19. Положение о выпускной квалификационной работе студентов, обучающихся по образовательным программам подготовки бакалавров: Введено в действие приказом РТУ

МИРЭА от 05.10.2018 № 1320, Внесены изменения приказом РТУ МИРЭА от 22.09.2022 № 1527. – URL: <https://www.mirea.ru/docs/177322> (дата обращения 10.05.2025). – Текст: электронный.

7. СМКО МИРЭА 7.5.1/03.П.30-19. Порядок проведения государственной итоговой аттестации по образовательным программам высшего образования – программам бакалавриата, программам специалитета и программам магистратуры: введено в действие приказом РТУ МИРЭА от 17.06.2019 № 965. – URL: <https://www.mirea.ru/docs/177338> (дата обращения 10.05.2025) – Текст: электронный.

8. Федеральный государственный образовательный стандарт высшего образования – бакалавриат по направлению подготовки 09.03.04 Программная инженерия (ФГОС ВО 3++): Приказ Министерства образования и науки РФ от 19 сентября 2017 г. N 920: Редакция приказом Минобрнауки России от 8 февраля 2021 г. № 83. – URL: https://fgosvo.ru/uploadfiles/FGOS%20VO%203++/Bak/090304_B_3_15062021.pdf (дата обращения: 10.05.2025). – Текст: электронный.

9. Tower Defense [Электронный ресурс]. – URL: <https://bigenc.ru/c/tower-defense-ac068b> (дата обращения 10.03.2025).

10. Принципы работы динамической корректировки сложности [Электронный ресурс]. – URL: <https://dtf.ru/gamedev/60372-principy-raboty-dinamicheskoi-korrektirovki-slozhnosti> (дата обращения 24.02.2025).

11. Chatfield T. Difficulty is the Point, Not the Problem // Wired, 2013. – URL: <https://www.wired.com/story/difficulty-in-video-games> (дата обращения 11.03.2025). – Текст: электронный.

12. Unity – что такое [Электронный ресурс]. – URL: <https://skyeng.ru/magazine/wiki/it-industriya/chto-takoe-unity/> (дата обращения 09.03.2025).

13. Для чего нужен REST API [Электронный ресурс]. – URL: <https://simple-blog.vercel.app/2020/12/13/rest-api/> (дата обращения 09.03.2025).

14. Машинное обучение агентов в Unity [Электронный ресурс]. – URL: <https://habr.com/ru/articles/454612/> (дата обращения 24.02.2025).
15. What is UML (Unified Modeling Language)? [Электронный ресурс]. – URL: <https://www.linkedin.com/pulse/what-umlunified-modeling-language-lakshmi-devi-sunkavalli?ysclid=m4mi7mpqp4319351406> (дата обращения 09.03.2025).
16. Что такое JSON [Электронный ресурс]. – URL: <https://habr.com/ru/articles/668684/> (дата обращения 10.03.2025).
17. Rollings A., Adams E.: Andrew Rollings and Ernest Adams on Game Design Chapter 7: Gameplay. – New Riders Press, 2014.
18. What is a Sprite Sheet? [Электронный ресурс]. – URL: https://jinhwaoh.github.io/GD_for_the_Web/spritesheet2.html (дата обращения 26.03.2025).
19. Effect and FX Pixel Part 3 [Электронный ресурс]. – URL: <https://bdragon1727.itch.io/effect-and-fx-pixel-part-3> (дата обращения 24.03.2025).
20. Компонент Line Renderer [Электронный ресурс]. – URL: <https://unityhub.ru/manual/class-LineRenderer> (дата обращения 24.02.2025).
21. Создание пользовательского интерфейса (UI) в Unity [Электронный ресурс] – URL: <https://sky.pro/wiki/gamedev/sozdanie-polzovatelskogo-interfejsa-ui-v-unity/> (дата обращения 01.05.2025).
22. Class Agent [Электронный ресурс]. – URL: <https://docs.unity3d.com/Packages/com.unity.ml-agents@2.0/api/Unity.MLAgents.Agent.html> (дата обращения 05.04.2025).
23. Training ML-Agents [Электронный ресурс]. – URL: <https://unity-technologies.github.io/ml-agents/Training-ML-Agents/> (дата обращения 05.04.2025).

24. Using TensorBoard to Observe Training [Электронный ресурс]. – URL: <https://unity-technologies.github.io/ml-agents/Using-Tensorboard/> (дата обращения 06.04.2025).

25. Диаграмма Ганта [Электронный ресурс]. – URL: <https://v8.1c.ru/platforma/diagramma-ganta/> (дата обращений 07.05.2025).

26. Производственный календарь на 2025 г. [Электронный ресурс]. – URL: https://nalog-nalog.ru/proizvodstvennyj_kalendar/2025-6/ (дата обращений 07.05.2025).

27. Стандарт ФСБУ 6/2020 «Основные средства» [Электронный ресурс]. – URL: https://www.kontur-extern.ru/info/45930-uchet_osnovnyh_sredstv_po_fsbu_6_2020 (дата обращений 07.05.2025).

28. НК РФ Статья 254. Материальные расходы, Федеральный закон от 05.08.2000 N 117-ФЗ (ред. от 13.12.2024) [Электронный ресурс]. – URL: https://www.consultant.ru/document/cons_doc_LAW_28165/0644a51c8d171aad7127867a97d0749ec20be875/ (дата обращений 07.05.2025).

29. Единый тариф (единые размеры) страховых взносов на ОПС, ОСС, ОМС для плательщиков, производящих выплаты физическим лицам, на 2025 г. [Электронный ресурс]. – URL: https://www.consultant.ru/document/cons_doc_LAW_93256/97a8d65271700b79d87a7edf33832b5f3ce0fd18/ (дата обращений 07.05.2025).

30. Приказ Минтруда России от 30.12.2016 N 851н (ред. от 29.08.2024) "Об утверждении Классификации видов экономической деятельности по классам профессионального риска" (Зарегистрировано в Минюсте России 18.01.2017 N 45279) [Электронный ресурс]. – URL: https://www.consultant.ru/document/cons_doc_LAW_211247/ (дата обращений 07.05.2025).

31. Ахмин М.С. Использование искусственного интеллекта для динамической корректировки сложности игры // Инновационная наука. – Уфа, 2025. – № 4-2. – С. 31–33. –

URL: <https://www.elibrary.ru/item.asp?id=81024053> (дата обращения:
21.05.2025). – Текст: электронный.

ПРИЛОЖЕНИЕ А

Гейм-дизайн документ

А.1 Общая информация

Название: Dynamic Defence.

Жанр: Tower Defence.

Краткое описание: 2D игра в жанре «Tower Defence» с динамической корректировкой сложности в процессе игры.

Целевая аудитория: любители стратегических и тактических игр, поклонники жанра TD.

Платформы: ПК на Windows.

А.2 Геймплей

Цель игрока – пережить наибольшее количество волн врагов на уровне. Уровень состоит из вражеской базы, базы игрока, пути между ними и платформ. Враги появляются на вражеской базе, и идут по пути в сторону базы игрока. Игрок должен ставить и улучшать башни на платформы, с целью остановить продвижение врагов. Если враг доходит до базы, она теряет некоторое количество здоровья, если количество здоровья базы упадет ниже нуля, игрок проиграет.

За убийство каждого врага, а также за прохождение волны в целом игрок получает монеты. Монеты нужно тратить на покупку башен и их прокачку. Также монеты можно получить за продажу ненужной башни.

Раз в единицу времени модуль динамической корректировки сложности оценивает уровень мастерства игрока, и, в зависимости от него, изменяет параметры игры.

А.3 Геймплейный цикл

Основной геймплейный представлен на рисунке А.1. Сначала появляется волна, игрок ее побеждает, получает за это монеты и готовится к следующей волне (строит новые башни или улучшает предыдущие).

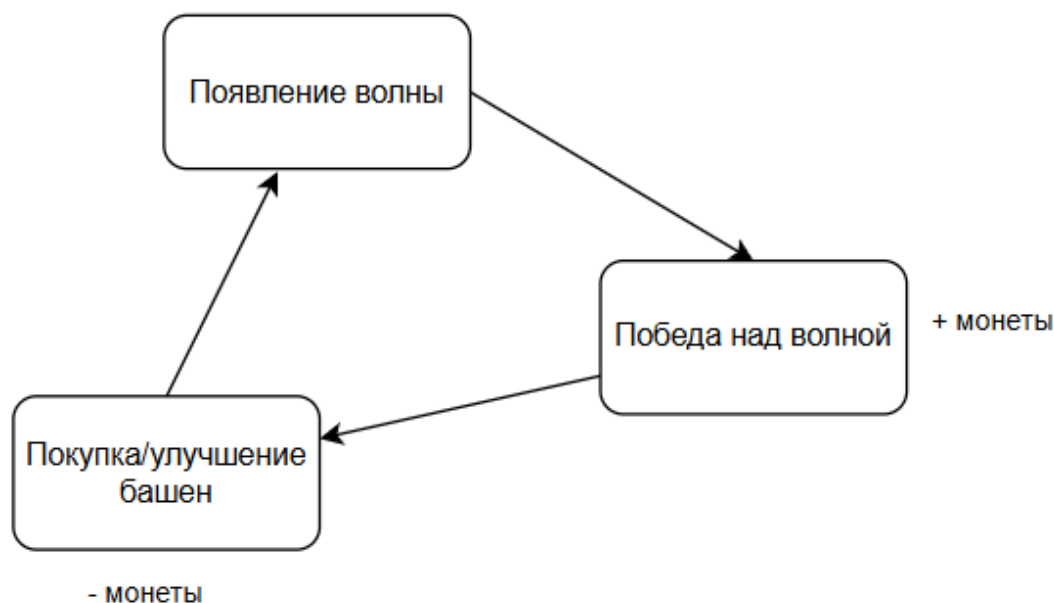


Рисунок А.1 – Основной геймплейный цикл [разработано автором]

А.4 Башни

В игре представлены следующие типы башен:

- Обычная башня – атакует одного врага за единицу времени.
- Взрывная башня – отличается от Обычной башни тем, наносит урон по области (эффективна против групп врагов).
- Ядовитая башня – отличается от Обычной башни тем, что накладывает врагам эффект отравления, наносящий урон со временем.
- Морозная башня – замедляет всех врагов в радиусе своего действия, но не наносит урона.

Башни имеют сбалансированные между собой характеристики, чтобы среди них не было явно лучшей.

Башни имеют несколько режимов выбора цели: первый враг; последний враг; враг с наибольшим количеством здоровья; враг с наименьшим количеством здоровья; рандомный враг.

Улучшение башен производится увеличением характеристик на некоторый процент. Башни имеют некоторый максимальный уровень.

A.5 Враги

В игре представлены следующие типы врагов:

- Обычный враг – сбалансированный по характеристикам враг.
- Быстрый враг – имеет большую скорость, но меньшее количество здоровья относительно Обычного врага.
- Тяжелый враг – имеет меньшую скорость, но большее количество здоровья относительно Обычного врага.

A.6 Графическое оформление

Игра представлена в минималистической 2D графике с простыми спрайтами и минимальными анимациями. В основном для графического оформления игры необходимы спрайты башен, врагов и элементов окружения (базы, платформы).

A.7 Звуковое оформление

В игре присутствует фоновая музыка и звуковые эффекты для погружения игрока в атмосферу игры.

A.8 Пользовательский интерфейс

Игра выводит основную информацию об игре посредством UI-элементов:

- В углу находится информация о текущем количестве здоровья базы и о текущем количестве монет у игрока.
- При получении урона у врагов появляется плашка с уровнем здоровья.
- При нажатии на башню открывается панель с ее основными характеристиками (например, урон, радиус и так далее, набор зависит от типа башни). В панели также можно улучшить или продать башню, а также выбрать режим выбора цели.
- При нажатии на вражескую базу открывается панель, в которой отображаются текущие характеристики врагов.

А.9 Вывод

Гейм-дизайн документ структурирует концепцию игры Dynamic Defence и служит основным руководством для разработчика. В нем описаны механики, динамическая корректировка сложности, игровой цикл, враги, башни, графическое и звуковое оформление, а также интерфейс. Данный документ позволяет организовать процесс разработки и избежать несоответствий в проектировании и разработки игры.

ПРИЛОЖЕНИЕ Б

Реализация игрового продукта

Листинг Б.1 – Класс CameraMove [разработано автором]

```
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.InputSystem.Controls;

public class CameraMove : MonoBehaviour
{
    public GameObject viewField;

    private bool isPressed;
    private Vector3 prevMousePosition;
    private Camera cam;
    private static float minCamSize = 3f;
    private static float maxCamSize = 10f;

    // Start is called once before the first execution of
    // Update after the MonoBehaviour is created
    void Start()
    {
        prevMousePosition = Input.mousePosition;
        cam = GetComponent<Camera>();
    }
    // Update is called once per frame
    void Update()
    {
        if (EventSystem.current.IsPointerOverGameObject())
        {
            isPressed = false;
            return;
        }

        float mouseScroll = Input.GetAxis("Mouse ScrollWheel");
        float scrollSpeed = 3f;
        if ((mouseScroll < 0) && (cam.orthographicSize -
mouseScroll*scrollSpeed < maxCamSize))
        {
            cam.orthographicSize -= mouseScroll * scrollSpeed;
        }
        if ((mouseScroll > 0) && (cam.orthographicSize -
mouseScroll*scrollSpeed > minCamSize))
        {
            cam.orthographicSize -= mouseScroll * scrollSpeed;
            float cameraHeight = 2f * cam.orthographicSize;
            float cameraWidth = cameraHeight * cam.aspect;
            Vector3 delta = new Vector3((Input.mousePosition.x
- Screen.width/2) / Screen.width * cameraWidth,
```

Продолжение листинга Б.1

```
(Input.mousePosition.y
- Screen.height/2) / Screen.height * cameraHeiht, 0);
    Vector3 newPosition = transform.position + delta /
((cam.orthographicSize - minCamSize) / mouseScroll);
    if (isInViewField(newPosition))
    {
        transform.position = newPosition;
    }
}
if (Input.GetKeyDown(KeyCode.Mouse0))
{
    isPressed = true;
}
if (Input.GetKeyUp(KeyCode.Mouse0)){isPressed = false;}
if (isPressed)
{
    float cameraHeiht = 2f * cam.orthographicSize;
    float cameraWidth = cameraHeiht * cam.aspect;
    Vector3 delta = Input.mousePosition -
prevMousePosition;
    Vector3 newPosition = transform.position - new
Vector3(delta.x * cameraWidth / Screen.width,
delta.y * cameraHeiht / Screen.height,
0);

    if (isInViewField(newPosition))
    {
        transform.position = newPosition;
    }
    prevMousePosition = Input.mousePosition;
}
bool isInViewField(Vector3 v)
{
    if ( v.x < (viewField.transform.position.x -
viewField.transform.localScale.x / 2) ||
        v.x > (viewField.transform.position.x +
viewField.transform.localScale.x / 2) ||
        v.y < (viewField.transform.position.y -
viewField.transform.localScale.y / 2) ||
        v.y > (viewField.transform.position.y +
viewField.transform.localScale.y / 2) )
    {
        return false;
    }
    return true;
}
}
```

Листинг Б.2 – Класс EnemySpawn [разработано автором]

```
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using UnityEngine;

public class EnemySpawn : MonoBehaviour
{
    public GameObject[] enemiesPrefab;
    //private int waveMax = 50;
    private int waveNum = 0;
    private float deltaWave = 15f;
    private int enemyCount = 8;
    private LineRenderer line;
    private Base mainBase;
    private float deltaCount;
    private float wavePower = 0.80f;
    private int waveAward = 25;
    private Dictionary<int, bool[]> waves = new Dictionary<int,
bool[]>();
    private ArrayList enemies = new ArrayList();
    private Dictionary<int, float[]> MLData = new
Dictionary<int, float[]>(); // number of wave : [waveLiveTime,
waveType]

    // Start is called once before the first execution of
Update after the MonoBehaviour is created
    void Start()
    {
        line =
GameObject.Find("MainPath").GetComponent<LineRenderer>();
        mainBase =
GameObject.Find("Base").GetComponent<Base>();
        mainBase.updateWaveText("Wave " + waveNum);

        transform.position = line.GetPosition(0) + new
Vector3(0, 0, -5);

        deltaCount = 5f;
    }

    // Update is called once per frame
    void Update()
    {
        foreach (int wn in MLData.Keys.ToList())
        {
            MLData[wn][0] += Time.deltaTime;
        }
        mainBase.updateWaveUnderText("Next wave in " +
(deltaWave - deltaCount).ToString("F1") + "s");
    }
}
```

Продолжение листинга Б.2

```
        foreach (int wn in waves.Keys.ToList()) {
            if (waves[wn][0])
            {
                continue;
            }
            bool b = true;
            for (int i = 0; i < enemies.Count; i++)
            {
                if ( (GameObject)enemies[i]!=null && wn ==
((GameObject)enemies[i]).GetComponent<Enemy>().getWaveNum() )
                {
                    b = false;
                    break;
                }
            }
            if (b) // окончание волны
            {

//GameObject.Find("MLPlayer").GetComponent<MLPlayer>().setMLData
a(MLData[wn][0], (int)MLData[wn][1]);

//GameObject.Find("MLPlayer").GetComponent<MLPlayer>().endWave(
waves[wn][1]);

GameObject.Find("MLDDA").GetComponent<MLDDA>().setMLData(MLData
[wn][0], (int)MLData[wn][1]);

GameObject.Find("MLDDA").GetComponent<MLDDA>().endWave(waves[wn
][1]);

                if (waves[wn][1])
                {
                    mainBase.addMoney(waveAward);
                }
                waves.Remove(wn);
                MLData.Remove(wn);

            }
        }
        if ((deltaCount < deltaWave) && (waves.Count != 0 ||
waveNum==0))
        {
            deltaCount += Time.deltaTime;
        }
    else
    {
        waveNum++;
        waves.Add(waveNum, new[] { true, true });
        MLData.Add(waveNum, new[] { 0f, 0f });
        StartCoroutine(Wave());
    }
}
```

Продолжение листинга Б.2

```
        deltaCount = 0f;

        for (int i = 0; i < enemies.Count; i++)
        {
            if (enemies[i]==null)
            {
                enemies.Remove(i);
                i--;
            }
        }
    }
}

IEnumerator Wave()
{
    if (waveNum % 5 == 0 && waveNum!=0)
    {
        enemyCount++;
        waveAward += 5;
    }

    mainBase.updateWaveText("Wave " + waveNum);

    int num = Random.Range(0, enemiesPrefab.Length);
    MLData[waveNum][1] = num;
    float delta = 0f;

    int count = 0;
    float timeCount = 0f;
    while (count < enemyCount)
    {
        if (timeCount > delta)
        {
            timeCount = 0;
            GameObject gm =
Instantiate(enemiesPrefab[num]);
            gm.GetComponent<Enemy>().setEnemy(wavePower,
waveNum);
            enemies.Add(gm);
            delta = -0.1f *
gm.GetComponent<Enemy>().getSpeed() + 0.4f;
            count++;
        }
        else
        {
            timeCount += Time.deltaTime;
        }
        yield return null;
    }
}
```

Продолжение листинга Б.2

```
        waves[waveNum][0] = false;
        wavePower += 0.1f;
    }
    public void offAward(int WaveNumber)
    {
        foreach (int wn in waves.Keys)
        {
            if (wn == WaveNumber)
            {
                waves[wn][1] = false;
                return;
            }
        }
    }
    public ArrayList getEnemies()
    {
        for (int i = 0; i < enemies.Count; i++)
        {
            if ((GameObject)enemies[i] == null)
            {
                enemies.Remove(i);
                i--;
            }
        }
        return enemies;
    }
    public void setDelta(int enemyCountDelta, int
waveAwardDelta, float wavePowerDelta)
    {
        enemyCount += enemyCountDelta;
        waveAward += waveAwardDelta;
        wavePower += wavePowerDelta;
    }
    public int getWaveNum()
    {
        return waveNum;
    }
    public int getEnemyCount()
    {
        return enemyCount;
    }
    public int getWaveAward()
    {
        return waveAward;
    }
    public float getWavePower()
    {
        return wavePower;
    }
}
```

Листинг Б.3 – Класс SaveFileHandler [разработано автором]

```
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;
using Newtonsoft.Json;

public static class SaveFileHandler
{
    private static string PATH = Application.persistentDataPath
+ "/save.json";

    public static void Save(DataStruct data)
    {
        string json = JsonConvert.SerializeObject(data,
Formatting.None);
        File.WriteAllText(PATH, json);
    }

    public static void CreateNewSaveIfNotExists()
    {
        if (!File.Exists(PATH))
        {
            DataStruct data = new DataStruct();
            data.musicVolume = 0.25f;
            data.soundVolume = 1.0f;
            data.levelRecords = new Dictionary<string, int>();

            string json = JsonConvert.SerializeObject(data,
Formatting.None);
            File.WriteAllText(PATH, json);
        }
    }

    public static DataStruct Load()
    {
        if (File.Exists(PATH))
        {
            string json = File.ReadAllText(PATH);
            return
JsonConvert.DeserializeObject<DataStruct>(json);
        }
        return null;
    }
}
```


Листинг Б.4 – Класс SaveManager [разработано автором]

```
using System.Collections.Generic;
public static class SaveManager
{
    public static float[] getVolume()
    {
        DataStruct data = SaveFileHandler.Load();
        return new float[] { data.musicVolume ,
data.soundVolume} ;
    }
    public static void setVolume(float musicVolume, float
soundVolume)
    {
        DataStruct data = SaveFileHandler.Load();
        data.musicVolume = musicVolume;
        data.soundVolume = soundVolume;
        SaveFileHandler.Save(data);
    }
    public static int getLevelRecord(string level)
    {
        DataStruct data = SaveFileHandler.Load();
        if (data.levelRecords.ContainsKey(level))
        {
            return data.levelRecords[level];
        }
        else
        {
            return -1;
        }
    }
    public static Dictionary<string, int> getLevelRecords()
    {
        DataStruct data = SaveFileHandler.Load();
        return data.levelRecords;
    }
    public static void setLevelRecord(string level, int record)
    {
        DataStruct data = SaveFileHandler.Load();
        if (data.levelRecords.ContainsKey(level))
        {
            data.levelRecords[level] = record;
        }
        else
        {
            data.levelRecords.Add(level, record);
        }
        SaveFileHandler.Save(data);
    }
}
```

ПРИЛОЖЕНИЕ В

Реализация моделей ИИ

Листинг В.1 – Класс MLDDA [разработано автором]

```
using UnityEngine;
using Unity.MLAgents;
using Unity.MLAgents.Actuators;
using Unity.MLAgents.Sensors;
using System.Collections.Generic;

public class MLDDA : Agent
{
    private List<Vector3> lastHitPositions = new
List<Vector3>();
    private LineRenderer line;
    private EnemySpawn enemySpawn;
    private bool isChange = false;
    private float pathDistance;
    private static int numObservations = 5;
    private List<int> waveEnemyTypeList = new
List<int>();
    private List<float> waveLiveTimeList = new
List<float>();
    private List<float> lastDistanceList = new
List<float>();
    private List<float> lastHitDistanceList = new
List<float>();
    private List<float> meanLastHitDistanceList = new
List<float>();

    void Start()
    {
        line =
GameObject.Find("MainPath").GetComponent<LineRenderer>();
        enemySpawn =
GameObject.Find("EnemySpawn").GetComponent<EnemySpawn>();
        pathDistance =
calculatePathDistance(line.GetPosition(line.positionCount-1));
    }
    public override void CollectObservations(VectorSensor
sensor)
    {
        sensor.AddObservation(isChange);
        sensor.AddObservation(pathDistance);

        //sensor.AddObservation(enemySpawn.getWaveNum());
        sensor.AddObservation(enemySpawn.getEnemyCount());
        sensor.AddObservation(enemySpawn.getWaveAward());
    }
}
```

Продолжение листинга В.1

```
        sensor.AddObservation(enemySpawn.getWavePower());

        for (int i = 0; i < numObservations; i++)
        {
            if (waveLiveTimeList.Count > i)
            {
                //sensor.AddObservation(waveLiveTimeList[i]);
                sensor.AddObservation(waveEnemyTypeList[i]);
                sensor.AddObservation(lastDistanceList[i]);
                sensor.AddObservation(lastHitDistanceList[i]);

//sensor.AddObservation(meanLastHitDistanceList[i]);
            }
            else
            {
                for (int j = 0; j < 3; j++)
                {
                    sensor.AddObservation(-1);
                }
            }
        }

        public override void OnActionReceived(ActionBuffers
actions)
        {
            if (isChange)
            {
                int enemyCountDelta = actions.DiscreteActions[0] -
1; //3
                int waveAwardDelta = actions.DiscreteActions[1] -
5; //11
                float wavePowerDelta = actions.ContinuousActions[0]
* 0.5f;
                enemySpawn.setDelta(enemyCountDelta,
waveAwardDelta, wavePowerDelta);
                Debug.Log(enemyCountDelta + " " + waveAwardDelta +
" " + wavePowerDelta + " \t\t\tReward: " +
GetCumulativeReward());
                isChange = false;
            }
        }

        public void addLastHitPosition(Vector3 pos)
        {
            lastHitPositions.Add(pos);
        }
```

Продолжение листинга В.1

```
public void setMLData(float lt, int et)
{
    addToList(waveLiveTimeList, lt);
    addToList(waveEnemyTypeList, et);
}

public void endWave(bool b)
{
    addToList(lastDistanceList, towersLastDistance());
    if (lastHitPositions.Count == 0)
    {
lastHitPositions.Add(line.GetPosition(line.positionCount - 1));
    }
    if (!b) addToList(lastHitDistanceList, pathDistance);
    else addToList(lastHitDistanceList,
calculatePathDistance(lastHitPositions[lastHitPositions.Count -
1]));
    float temp = 0;
    foreach (Vector3 pos in lastHitPositions)
    {
        temp += calculatePathDistance(pos);
    }
    addToList(meanLastHitDistanceList, temp /
lastHitPositions.Count);

    float ld = lastDistanceList[lastDistanceList.Count -
1];
    float lhd =
lastHitDistanceList[lastHitDistanceList.Count - 1];

    if (!b)
    {
        SetReward(-5f);
        Debug.Log("-0.5");
    }
    SetReward((-Mathf.Abs(ld - lhd) / pathDistance + 0.3f)
* 10);
    Debug.Log(-Mathf.Abs(ld - lhd) / pathDistance + 0.3f);

    lastHitPositions = new List<Vector3>();
    isChange = true;
}

float towersLastDistance()
{
    float step = 0.1f;
    float ans = 0;
    float levelSum = 0;
```

Продолжение листинга В.1

```
        foreach (GameObject gm in
GameObject.FindGameObjectsWithTag("Tower"))
        {
            Tower t = gm.GetComponent<Tower>();
            if (t.getTowerNum() == 3) continue;
            Vector3 point = line.GetPosition(0);
            int iter = 1;
            float path = 0;
            float longestTowerPath = 0;
            while (path < 1000f)
            {
                if (t.isInRadius(point))
                {
                    longestTowerPath = path;
                }

                float dist = Mathf.Pow((Mathf.Pow(point.x -
line.GetPosition(iter).x, 2) + Mathf.Pow(point.y -
line.GetPosition(iter).y, 2)), 0.5f);
                if (dist <= step/2)
                {
                    iter++;
                    if (iter == line.positionCount) break;
                    dist =
Mathf.Pow((Mathf.Pow(line.GetPosition(iter).x - point.x, 2) +
Mathf.Pow(line.GetPosition(iter).y - point.y, 2)), 0.5f);

                }
                float moveX = (line.GetPosition(iter).x -
point.x) / dist;
                float moveY = (line.GetPosition(iter).y -
point.y) / dist;
                point += new Vector3(moveX, moveY, 0) * step;
                path += step;
            }

            ans += longestTowerPath * t.getLevel();
            levelSum += t.getLevel();
        }
        if (levelSum == 0) return 0;
        else return ans / levelSum;
    }

    float calculatePathDistance(Vector3 pos)
    {
        float step = 0.1f;

        Vector3 point = line.GetPosition(0);
        int iter = 1;
```

Продолжение листинга В.1

```
        float path = 0;
        while (path < 1000f)
        {
            if (Mathf.Pow((Mathf.Pow(point.x - pos.x, 2) +
Mathf.Pow(point.y - pos.y, 2)), 0.5f) < 0.2f + step)
            {
                return path;
            }

            float dist = Mathf.Pow((Mathf.Pow(point.x -
line.GetPosition(iter).x, 2) + Mathf.Pow(point.y -
line.GetPosition(iter).y, 2)), 0.5f);
            if (dist <= step/2)
            {
                iter++;
                if (iter == line.positionCount) break;
                dist =
Mathf.Pow((Mathf.Pow(line.GetPosition(iter).x - point.x, 2) +
Mathf.Pow(line.GetPosition(iter).y - point.y, 2)), 0.5f);
            }
            float moveX = (line.GetPosition(iter).x - point.x)
/ dist;
            float moveY = (line.GetPosition(iter).y - point.y)
/ dist;
            point += new Vector3(moveX, moveY, 0) * step;
            path += step;
        }
        return path;
    }
    public void endEpisodeVoid()
    {
        //SetReward(GetCumulativeReward() /
enemySpawn.getWaveNum());

        EndEpisode();
    }
    void addToList<T>(List<T> list, T val)
    {
        if (list.Count == numObservations)
        {
            list.RemoveAt(0);
            list.Add(val);
        }
        else
        {
            list.Add(val);
        }
    }
}
```

ПРИЛОЖЕНИЕ Г

Диаграмма Ганта

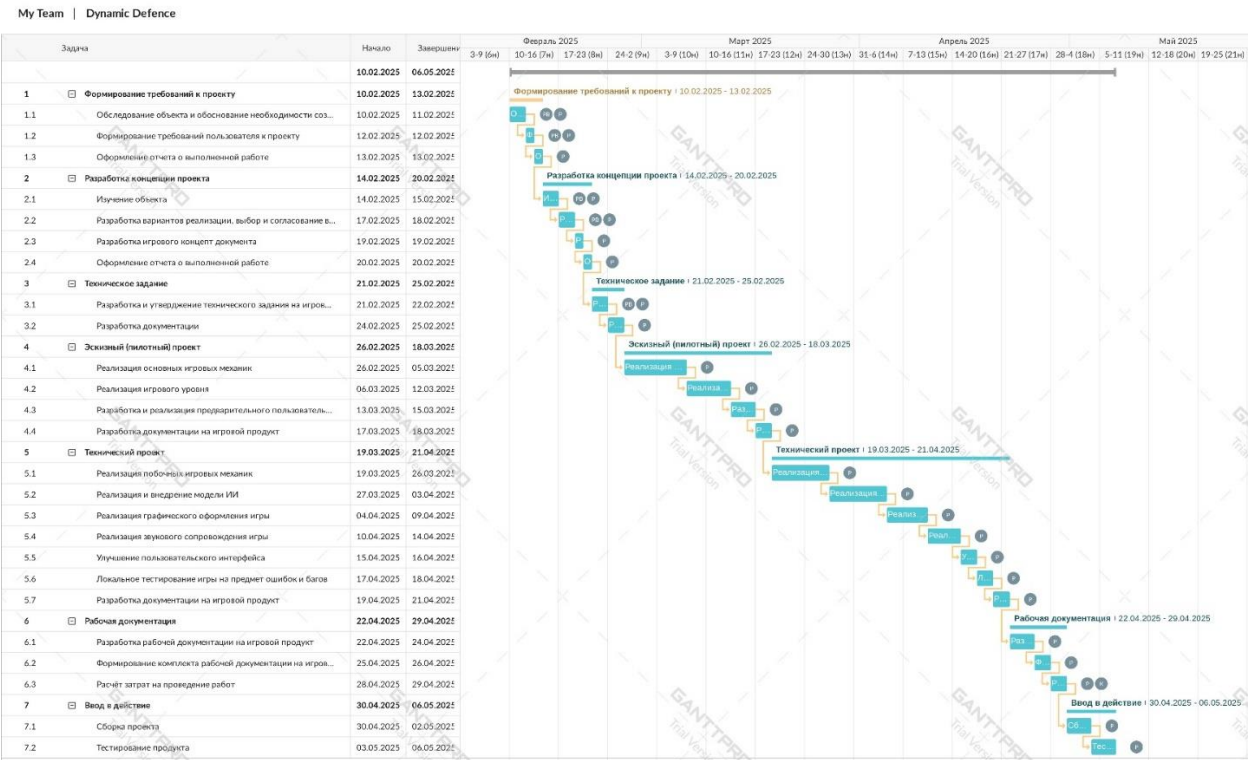


Рисунок Г.1 – Календарный график выполнения работ [разработано автором]